action

Action tags come from single-shot regexes over the combined text buffer: tweet text plus OCR text, transcripts, and media-description text. One match emits the corresponding action tag with a source span.

scripts/tag_lexical.py:1518-1591

```
    # layer); needs-vision means the frame/scene itself has not been analyzed
    # (resolved only by an actual visual description, never by OCR).
    if needs_ocr:
        add("media-status:needs-ocr", tentative=True, source="media-metadata")

    # Concatenate OCR and media-description text so a poster's stamped
    # slogan or manually reviewed image description earns the same tags
    # as if the words had been typed into the tweet body.
    body_parts = [part for part in (text, ocr_text, transcript_text, media_text) if part]
    body = " || ".join(body_parts)

    if not body:
        # Even with no text we still get format: + agency: + sticky
        # default. Skip the regex pass.
        _ensure_intrinsic_parent_topics(entries, "", add)
        _maybe_immigration_default(entries, account_category, "", add)
        return entries
    text = body

    # Single-shot regex tags
    for pat, tag in (
        (PATTERN_FRAME_CRIMINAL, "theme:criminal"),
        (PATTERN_ACTION_DETENTION, "action:detention"),
        (PATTERN_ACTION_SELF_DEPORTATION, "action:self-deportation"),
        (PATTERN_ACTION_DEPORTATION, "action:deportation"),
        (PATTERN_ACTION_REPORT_TO_ICE, "action:report-immigrants"),
        (PATTERN_SUBJECT_CBP_HOME_APP, "subject:cbp-home-app"),
        (PATTERN_SUBJECT_CELEBRITY, "subject:celebrity"),
        (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
        (PATTERN_LEGAL_CIVIL_LAWSUIT, "legal:civil-lawsuit"),
        (PATTERN_TOPIC_ECONOMY, "topic:economy"),
        (PATTERN_TOPIC_MILITARY, "topic:military"),
        (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),
        (PATTERN_EVENT_PALESTINE, "event:palestine"),
        (PATTERN_THEME_BORDER, "policy:border"),
        (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
        (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
        (PATTERN_THEME_HOMELAND, "theme:homeland"),
        (PATTERN_THEME_NATIVISM, "theme:nativism"),
        (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
        (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
        (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
        (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
        (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
        (PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP, "legal:birthright-citizenship"),
        (PATTERN_THEME_NATIVISM_BIRTHRIGHT, "theme:nativism"),
        (PATTERN_SLOGAN_NICE, "slogan:nice"),
        (PATTERN_SLOGAN_WORST, "slogan:worst"),
        (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
        (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
        (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
        (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
        (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
        (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homecoming"),
        (PATTERN_SLOGAN_MAGA, "slogan:maga"),
        (PATTERN_SLOGAN_MAHA, "slogan:maha"),
        (PATTERN_SLOGAN_MASA, "slogan:masa"),
        (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
        (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
        (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
        (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
        (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
        (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
        (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
        (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
        (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
        (PATTERN_SLOGAN_FIND_AND_KILL, "slog
... [truncated in PDF; see source file for full pattern]
```

scripts/tag_lexical.py:515-559

```
PATTERN_FRAME_CRIMINAL = _compile(
    r"\b(criminal illegal aliens?|illegal aliens?|criminal aliens?|aggravated felons?|convicted (?:for|of))\b"
)
PATTERN_ACTION_DETENTION = _compile(
    r"\b(arrest(?:ed|ing)?|detain(?:ed|ing)?|apprehend(?:ed|ing)?|in custody|nab(?:bed)?)\b"
)
PATTERN_ACTION_SELF_DEPORTATION = _compile(
    r"\bself[- ]deport(?:ed|ing|ation)?\b"
    r"|\bvoluntar(?:y|ily)\s+depart(?:ure|ed|ing)?\b"
    r"|\bleave voluntarily\b"
    r"|\breport\s+(?:your|their|his|her|an|the)?\s*departure\b"
    r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
)
PATTERN_ACTION_DEPORTATION = _compile(
    r"(?<!self-)(?<!self )\b(deport(?:ed|ing|ation)?|remov(?:ed|al)|repatriat(?:ed|ion))\b"
)
ICE_OR_DHS_TARGET = (
    r"(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|"
    r"U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement|DHS|"
    r"Department of Homeland Security|Homeland Security)"
)
ICE_DIRECT_TARGET = (
    r"(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|"
    r"U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement)"
)
ICE_REPORT_SUBJECT = (
```

```
    r"(?:illegal\s+(?:alien|immigrant)s?|criminal\s+aliens?|"
    r"undocumented\s+(?:immigrant|alien)s?|immigration\s+(?:crime|violation)s?|"
    r"alien(?:s|['\u2019]s)?|immigrants?|migrants?)"
)
ICE_REPORT_PHONE = r"(?:866[-\s]?DHS[-\s]?2[-\s]?ICE|866[-\s]?347[-\s]?2423)"
PATTERN_ACTION_REPORT_TO_ICE = _compile(
    rf"\b(?:"
    rf"(?:call|contact)\s+(?:{ICE_DIRECT_TARGET}|{ICE_REPORT_PHONE})\b"
    rf"(?=.{{0,180}}\b{ICE_REPORT_SUBJECT}\b)"
    rf"|(?:report|tip|submit|send|notify)\b.{{0,80}}\b{ICE_REPORT_SUBJECT}\b"
    rf".{{0,80}}\b(?:to|at|with)\s+(?:{ICE_OR_DHS_TARGET}|{ICE_REPORT_PHONE})\b"
    rf"|(?:submit|send)\s+(?:a\s+)?tip\b.{{0,80}}\b(?:to|with)\s+"
    rf"(?:{ICE_OR_DHS_TARGET}|{ICE_REPORT_PHONE})\b"
    rf"(?=.{{0,120}}\b{ICE_REPORT_SUBJECT}\b)"
    rf"|{ICE_REPORT_SUBJECT}\b.{{0,240}}\b(?:call|contact)\s+"
    rf"(?:(?:our|the)\s+)?(?:{ICE_DIRECT_TARGET}|ICE\s+)?"
    rf"(?:tip\s*line|hotline|{ICE_REPORT_PHONE})\b"
    rf")"
)
```

Tag	Count	How it is produced
`action:detention`	2,278 (22.8%)	Config pattern in config/tag_taxonomy.yaml: \b(arrest(?:ed|ing)?|detain(?:ed|ing)?|apprehend(?:ed|ing)?|in custody|nab(?:bed)?)\b
`action:deportation`	1,304 (13.0%)	Config pattern in config/tag_taxonomy.yaml: \b(deport(?:ed|ing|ation)?|remov(?:ed|al)|repatriat(?:ed|ion))\b
`action:self-deportation`	191 (1.9%)	Config pattern in config/tag_taxonomy.yaml: \b(self[-]deport(?:ed|ing|ation)?|voluntar(?:y|ily) depart(?:ure|ed|ing)?|leave voluntarily|report (?:your|their|his|her)? departure)\b
`action:report-immigrants`	7 (0.1%)	Config pattern in config/tag_taxonomy.yaml: \b(?:(?:call|contact)\s+(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement|866[-\s]?DHS[-\s]?2[-\s]?ICE|866[-\s]?347[-\s]?2423)\b(?=.{0,180}\b(?:illegal\s+(?:alien|immigrant)s?|criminal\s+aliens?|undocumented\s+(?:immigrant|alien)s?|immigration\s+(?:crime|violation)s?|alien(?:s|['']s)?|immigrants?|migrants?)\b)|(?:report|tip|submit|send|notify)\b.{0,80}\b(?:illegal\s+(?:alien|immigrant)s?|criminal\s+aliens?|undocumented\s+(?:immigrant|alien)s?|immigration\s+(?:crime|violation)s?|alien(?:s|['']s)?|immigrants?|migrants?)\b.{0,80}\b(?:to|at|with)\s+(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement|DHS|Department of Homeland Security|Homeland Security|866[-\s]?DHS[-\s]?2[-\s]?ICE|866[-\s]?347[-\s]?2423)\b|(?:submit|send)\s+(?:a\s+)?tip\b.{0,80}\b(?:to|with)\s+(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement|DHS|Department of Homeland Security|Homeland Security|866[-\s]?DHS[-\s]?2[-\s]?ICE|866[-\s]?347[-\s]?2423)\b(?=.{0,120}\b(?:illegal\s+(?:alien|immigrant)s?|criminal\s+aliens?|undocumented\s+(?:immigrant|alien)s?|immigration\s+(?:crime|violation)s?|alien(?:s|['']s)?|immigrants?|migrants?)\b)|(?:illegal\s+(?:alien|immigrant)s?|criminal\s+aliens?|undocumented\s+(?:immigrant|alien)s?|immigration\s+(?:crime|violation)s?|alien(?:s|['']s)?|immigrants?|migrants?)\b.{0,240}\b(?:call|contact)\s+(?:(?:our|the)\s+)?(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement|ICE\s+)?(?:tip\s*line|hotline|866[-\s]?DHS[-\s]?2[-\s]?ICE|866[-\s]?347[-\s]?2423)\b)

artist

Artist tags are not inferred from audio. They fire only when the artist name or one of the listed signature song phrases appears in tweet text, OCR, transcript, or media-description text.

scripts/tag_lexical.py:939-959

```
# — right now artist names are almost entirely in the (un-OCR'd) images / audio,
# not the tweet bodies.
ARTIST_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (
    (_compile(r"\bsydney\s+sweeney\b"), "sydney-sweeney"),
    (
        _compile(
            r"\blee\s+greenwood\b|\bgod\s+bless\s+the\s+u\.?s\.?a\.?\b"
            r"|\bproud\s+to\s+be\s+an\s+american\b"
        ),
        "lee-greenwood",
    ),
    (_compile(r"\bkid\s+rock\b"), "kid-rock"),
    (_compile(r"\bvillage\s+people\b|\bmacho\s+man\b"), "village-people"),
    (_compile(r"\bjason\s+aldean\b|\btry\s+that\s+in\s+a\s+small\s+town\b"), "jason-aldean"),
    (_compile(r"\boliver\s+anthony\b|\brich\s+men\s+north\s+of\s+richmond\b"), "oliver-anthony"),
    (_compile(r"\btaylor\s+swift\b"), "taylor-swift"),
    (_compile(r"\bbeyonc[eé]\b"), "beyonce"),
)

# --- video:<kind> ------------------------------------------------------------
#
```

scripts/tag_lexical.py:1672-1675

```
    # artist:<name> — named cultural figures / musicians (and signature songs).
    for artist_pat, artist_slug in ARTIST_GAZETTEER:
        if m := artist_pat.search(text):
            add(f"artist:{artist_slug}", span=m.span())
```

Tag	Count	How it is produced
`artist:lee-greenwood`	4 (0.0%)	Slug lee-greenwood is emitted by ARTIST_GAZETTEER when the artist name or signature phrase is present in text/OCR/transcript/media-description text.

Tag	Count	How it is produced
`artist:kid-rock`	2 (0.0%)	Slug kid-rock is emitted by ARTIST_GAZETTEER when the artist name or signature phrase is present in text/OCR/transcript/media-description text.

country

Country tags are validated extractions. The code matches country-name candidates after limited prepositions, curated demonyms, and curated foreign city names, then resolves them against the country vocabulary before normalizing the slug.

scripts/tag_lexical.py:1217-1316

```
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!(?i:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," — anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?i:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?i:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" — anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
1231:
1232: # Demonyms and major foreign cities -> country. The validators above need a
1233: # preposition + the exact country name, so they miss "Iranian regime" (demonym)
1234: # and "in Tehran" (city) — the most common way countries actually appear. These
1235: # maps recover them; demonyms are distinctive enough to match without a
1236: # preposition. Curated for precision: food/ambiguous demonyms ("French",
1237: # "German", "Indian", "Korean") are intentionally omitted.
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
```

```
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person
    ... [truncated in PDF; see source file for full pattern]
```

scripts/tag_lexical.py:1683-1701

```
1683:     # country:<NAME> — broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))
```

scripts/tag_lexical.py:2046-2066

```
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

Tag	Count	How it is produced
country:Mexico	771 (7.7%)	Slug Mexico is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:El-Salvador	266 (2.7%)	Slug El-Salvador is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Guatemala	189 (1.9%)	Slug Guatemala is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Honduras	180 (1.8%)	Slug Honduras is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Iran	133 (1.3%)	Slug Iran is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Venezuela	133 (1.3%)	Slug Venezuela is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Cuba	100 (1.0%)	Slug Cuba is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:China	98 (1.0%)	Slug China is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Dominican-Republic	70 (0.7%)	Slug Dominican-Republic is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Colombia	54 (0.5%)	Slug Colombia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Laos	51 (0.5%)	Slug Laos is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Ecuador	49 (0.5%)	Slug Ecuador is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Israel	43 (0.4%)	Slug Israel is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Vietnam	39 (0.4%)	Slug Vietnam is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.

Tag	Count	How it is produced
country:India	36 (0.4%)	Slug India is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Somalia	35 (0.3%)	Slug Somalia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Haiti	33 (0.3%)	Slug Haiti is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Brazil	31 (0.3%)	Slug Brazil is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Canada	31 (0.3%)	Slug Canada is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Afghanistan	27 (0.3%)	Slug Afghanistan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Jamaica	25 (0.2%)	Slug Jamaica is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Russia	25 (0.2%)	Slug Russia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Nicaragua	22 (0.2%)	Slug Nicaragua is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Pakistan	15 (0.1%)	Slug Pakistan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Ukraine	15 (0.1%)	Slug Ukraine is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Georgia	14 (0.1%)	Slug Georgia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Nigeria	13 (0.1%)	Slug Nigeria is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Argentina	11 (0.1%)	Slug Argentina is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Peru	11 (0.1%)	Slug Peru is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Lebanon	10 (0.1%)	Slug Lebanon is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:United-States	10 (0.1%)	Slug United-States is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Egypt	8 (0.1%)	Slug Egypt is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Iraq	8 (0.1%)	Slug Iraq is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Panama	8 (0.1%)	Slug Panama is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Cambodia	7 (0.1%)	Slug Cambodia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Sierra-Leone	7 (0.1%)	Slug Sierra-Leone is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:South-Korea	7 (0.1%)	Slug South-Korea is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:France	6 (0.1%)	Slug France is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Guyana	6 (0.1%)	Slug Guyana is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Jordan	6 (0.1%)	Slug Jordan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Sudan	6 (0.1%)	Slug Sudan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Syria	6 (0.1%)	Slug Syria is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Ghana	5 (0.0%)	Slug Ghana is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Japan	5 (0.0%)	Slug Japan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Myanmar	5 (0.0%)	Slug Myanmar is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.

Tag	Count	How it is produced
country:Turkey	5 (0.0%)	Slug Turkey is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Barbados	4 (0.0%)	Slug Barbados is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Cameroon	4 (0.0%)	Slug Cameroon is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Chile	4 (0.0%)	Slug Chile is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Ethiopia	4 (0.0%)	Slug Ethiopia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Germany	4 (0.0%)	Slug Germany is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Greece	4 (0.0%)	Slug Greece is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Ireland	4 (0.0%)	Slug Ireland is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Kenya	4 (0.0%)	Slug Kenya is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Liberia	4 (0.0%)	Slug Liberia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Philippines	4 (0.0%)	Slug Philippines is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Qatar	4 (0.0%)	Slug Qatar is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Rwanda	4 (0.0%)	Slug Rwanda is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:South-Africa	4 (0.0%)	Slug South-Africa is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Trinidad	4 (0.0%)	Slug Trinidad is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Australia	3 (0.0%)	Slug Australia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Bahrain	3 (0.0%)	Slug Bahrain is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Bangladesh	3 (0.0%)	Slug Bangladesh is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Costa-Rica	3 (0.0%)	Slug Costa-Rica is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Italy	3 (0.0%)	Slug Italy is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Senegal	3 (0.0%)	Slug Senegal is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Spain	3 (0.0%)	Slug Spain is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Thailand	3 (0.0%)	Slug Thailand is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Uganda	3 (0.0%)	Slug Uganda is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Uzbekistan	3 (0.0%)	Slug Uzbekistan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Angola	2 (0.0%)	Slug Angola is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Armenia	2 (0.0%)	Slug Armenia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Azerbaijan	2 (0.0%)	Slug Azerbaijan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Burkina-Faso	2 (0.0%)	Slug Burkina-Faso is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Finland	2 (0.0%)	Slug Finland is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Gabon	2 (0.0%)	Slug Gabon is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.

Tag	Count	How it is produced
country:Kazakhstan	2 (0.0%)	Slug Kazakhstan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Kyrgyzstan	2 (0.0%)	Slug Kyrgyzstan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Portugal	2 (0.0%)	Slug Portugal is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Romania	2 (0.0%)	Slug Romania is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:South-Sudan	2 (0.0%)	Slug South-Sudan is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:United-Kingdom	2 (0.0%)	Slug United-Kingdom is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Uruguay	2 (0.0%)	Slug Uruguay is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Yemen	2 (0.0%)	Slug Yemen is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Algeria	1 (0.0%)	Slug Algeria is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Belarus	1 (0.0%)	Slug Belarus is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Belgium	1 (0.0%)	Slug Belgium is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Belize	1 (0.0%)	Slug Belize is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Botswana	1 (0.0%)	Slug Botswana is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Chad	1 (0.0%)	Slug Chad is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Congo	1 (0.0%)	Slug Congo is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Djibouti	1 (0.0%)	Slug Djibouti is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Dominica	1 (0.0%)	Slug Dominica is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Iceland	1 (0.0%)	Slug Iceland is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Latvia	1 (0.0%)	Slug Latvia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Mauritania	1 (0.0%)	Slug Mauritania is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Mauritius	1 (0.0%)	Slug Mauritius is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Montenegro	1 (0.0%)	Slug Montenegro is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Namibia	1 (0.0%)	Slug Namibia is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Poland	1 (0.0%)	Slug Poland is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Togo	1 (0.0%)	Slug Togo is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:Tonga	1 (0.0%)	Slug Tonga is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.
country:United-Arab-Emirates	1 (0.0%)	Slug United-Arab-Emirates is emitted only after a matched country candidate, demonym, or foreign-city cue resolves to that country in the validated country vocabulary.

crime

Crime tags are vocabulary-driven regex matches. Every distinct crime vocabulary match emits its own crime:<type> tag; some narrower crimes also emit parent buckets such as crime:sexual, crime:narcotics, crime:homicide, or crime:disobedience.

scripts/tag_lexical.py:288-331

```
288: # Vocabulary for `crime:<TYPE>`. Each entry is (slug, pattern). The slug
```

```
289: # becomes the tag suffix (`crime:assault`, `crime:fentanyl`, etc.).
290: CRIME_VOCAB: tuple[tuple[str, str], ...] = (
291:     ("rape", r"\brap(?:e|ed|ing|ist)\b"),
292:     ("sodomy", r"\bsodom(?:y|ize|ized)\b"),
293:     ("homicide", r"\bhomicides?\b"),
294:     ("burglary", r"\bburglar(?:y|ize|ized|ies)\b"),
295:     ("theft", r"\btheft\b|\bsteal(?:ing)?\b|\bstole\b"),
296:     ("robbery", r"\brobber(?:y|ies)\b|\brobbed\b"),
297:     ("assault", r"\bassault(?:ed|ing|s)?\b"),
298:     ("battery", r"\bbatter(?:y|ies|ed)\b"),
299:     ("fentanyl", r"\bfentanyl\b"),
300:     ("cocaine", r"\bcocaine\b"),
301:     ("meth", r"\bmethamphetamine\b|\bmeth\b"),
302:     ("trafficking", r"\btraffick(?:ing|er|ers)\b"),
303:     ("dui", r"\bDUI\b|\bDWI\b"),
304:     ("kidnap", r"\bkidnap(?:ped|ping|per)?\b"),
305:     ("sexual", r"\b(?:sexual\s+assault|sex\s+crime|sex\s+offen[sc]e|sex\s+abuse)\b"),
306:     (
307:         "child-sexual",
308:         r"\b(?:child\s+(?:sex(?:ual)?|pornography|molest(?:ation)?)|"
309:         r"child\s+sexual|minor\s+sex(?:ual)?|sex(?:ual)?\s+abuse\s+of\s+(?:a\s+)?minor)\b",
310:     ),
311:     ("child", r"\bchild (?:abuse|sex|pornography|molest)|child sexual\b"),
312:     ("gang", r"\bgang (?:member|members|affiliation)\b|\bgang-related\b"),
313:     ("ms13", r"\bMS-?13\b"),
314:     ("tren-de-aragua", r"\bTren de Aragua\b|\bTdA\b"),
315:     ("narcotics", r"\bnarcotic[s]?\b"),
316:     ("fraud", r"\bfraud(?:ulent|ster)?\b"),
317:     (
318:         "disobedience",
319:         r"\b(?:contempt\s+of\s+court|violat(?:e|ed|ing|ion)\s+(?:of\s+)?(?:a\s+)?court\s+order|disobey(?:ed|ing)?\s+(?:a\s+)?court\s+order)\b",
320:     ),
321:     ("perjury", r"\bperjur(?:y|ed)\b|\blying\s+under\s+oath\b"),
322:     ("arson", r"\barson(?:ist)?\b"),
323:     ("weapon", r"\bweapon[s]?\b|\billegal firearm[s]?\b"),
324:     ("firearm", r"\bfirearm[s]?\b"),
325: )
326:
327: CRIME_SUBTYPE_VOCAB: tuple[tuple[str, str], ...] = (("murder", r"\bmurder(?:s|ed|ers?|ing)?\b"),)
328:
329: CRIME_PARENT_TAGS: dict[str, tuple[str, ...]] = {
330:     "crime:murder": ("crime:homicide",),
331:     "crime:rape": ("crime:sexual",),
```

scripts/tag_lexical.py:1638-1653

```
1638:     # crime:<TYPE> — every distinct match emits one tag entry.
1639:     for slug, pat_str in CRIME_VOCAB:
1640:         for m in re.finditer(pat_str, text, re.I):
1641:             add(f"crime:{slug}", span=m.span())
1642:             for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:                 add(parent, span=m.span())
1644:     for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:         for m in re.finditer(pat_str, text, re.I):
1646:             tag = f"crime:{slug}"
1647:             add(tag, span=m.span())
1648:             for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:                 add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:
```

Tag	Count	How it is produced
crime:assault	911 (9.1%)	Slug assault is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:homicide	737 (7.4%)	Slug homicide is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:sexual	619 (6.2%)	Slug sexual is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:murder	614 (6.1%)	Slug murder is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:narcotics	516 (5.2%)	Slug narcotics is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:trafficking	405 (4.0%)	Slug trafficking is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:weapon	377 (3.8%)	Slug weapon is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:gang	337 (3.4%)	Slug gang is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:rape	276 (2.8%)	Slug rape is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:child	256 (2.6%)	Slug child is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:fraud	226 (2.3%)	Slug fraud is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:cocaine	222 (2.2%)	Slug cocaine is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.

Tag	Count	How it is produced
crime:child-sexual	215 (2.1%)	Slug child-sexual is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:battery	187 (1.9%)	Slug battery is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:firearm	177 (1.8%)	Slug firearm is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:theft	166 (1.7%)	Slug theft is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:robbery	164 (1.6%)	Slug robbery is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:fentanyl	156 (1.6%)	Slug fentanyl is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:ms13	156 (1.6%)	Slug ms13 is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:dui	148 (1.5%)	Slug dui is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:burglary	132 (1.3%)	Slug burglary is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:meth	132 (1.3%)	Slug meth is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:kidnap	123 (1.2%)	Slug kidnap is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:tren-de-aragua	117 (1.2%)	Slug tren-de-aragua is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:arson	29 (0.3%)	Slug arson is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:sodomy	23 (0.2%)	Slug sodomy is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:disobedience	8 (0.1%)	Slug disobedience is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.
crime:perjury	1 (0.0%)	Slug perjury is emitted from CRIME_VOCAB or CRIME_SUBTYPE_VOCAB. Parent crime buckets are added through CRIME_PARENT_TAGS when applicable.

event

Event tags are emitted either by a direct topic regex, such as Palestine/Gaza/Hamas language, or by curated city-plus-disturbance patterns that require both a place and unrest/federal-response language.

scripts/tag_lexical.py:639-644

```
639: PATTERN_EVENT_PALESTINE = _compile(
640:     r"\b(?:palestin(?:e|ian)s?|gaza(?:n)?s?|hamas|israel[- ]hamas|"
641:     r"israel[- ]palestin(?:e|ian)|west\s+bank)\b"
642: )
643: GENERAL_TOPIC_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
644:     ("immigration", _compile(r"\b(immigration|migrant|border|illegal alien|asylum)\b")),
```

scripts/tag_lexical.py:727-763

```
727:     r"violent\s+protests?|anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|"
728:     r"violent\s+demonstrators?|street\s+violence|mob\s+violence)\b"
729: )
730: DISTURBANCE_CITY_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
731:     (
732:         "los-angeles-disturbance",
733:         _compile(
734:             r"\b(?:Los\s+Angeles|L\.A\.|LA)\b.{0,180}\b(?:riot(?:s|ers|ing)?|"
735:             r"civil\s+disturbance|civil\s+unrest|violent\s+protests?|anti[- ]ICE\s+"
736:             r"(?:riot(?:s|ers|ing)?|protests?)|federal\s+building|concrete\s+at\s+DHS\s+agents)\b"
737:             r"|\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|violent\s+protests?|"
738:             r"anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|federal\s+building|"
739:             r"concrete\s+at\s+DHS\s+agents)\b.{0,180}\b(?:Los\s+Angeles|L\.A\.|LA)\b"
740:         ),
741:     ),
742:     (
743:         "minneapolis-disturbance",
744:         _compile(
745:             r"\bMinneapolis\b.{0,180}\b(?:Bovino|riot(?:s|ers|ing)?|civil\s+disturbance|"
746:             r"civil\s+unrest|violent\s+protests?|protests?|tear\s+gas|pellet\s+guns?|"
747:             r"heavy[- ]handed|lawsuits?|killing|killed|disaster)\b"
748:             r"|\b(?:Bovino|riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|"
749:             r"violent\s+protests?|protests?|tear\s+gas|pellet\s+guns?|heavy[- ]handed|"
750:             r"lawsuits?|killing|killed|disaster)\b.{0,180}\bMinneapolis\b"
751:         ),
752:     ),
753:     (
754:         "portland-disturbance",
755:         _compile(
```

```
 756:             r"\bPortland\b.{0,180}\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|"
 757:             r"violent\s+protests?|anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|"
 758:             r"ICE\s+(?:detention|facility)|Antifa|domestic\s+terrorism)\b"
 759:             r"|\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|violent\s+protests?|"
 760:             r"anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|ICE\s+(?:detention|facility)|"
 761:             r"Antifa|domestic\s+terrorism)\b.{0,180}\bPortland\b"
 762:         ),
 763:     ),
```

scripts/tag_lexical.py:1609-1612

```
1609:     if m := _coded_nativism_match(text, entries):
1610:         add("theme:nativism", span=m.span())
1611:
1612:     if m := _theme_religion_match(text):
```

Tag	Count	How it is produced
`event:palestine`	36 (0.4%)	Emitted when PATTERN_EVENT_PALESTINE matches the combined text buffer.
`event:los-angeles-disturbance`	15 (0.1%)	Slug los-angeles-disturbance is emitted by DISTURBANCE_CITY_PATTERNS when the named city appears near riot/unrest/protest/federal-response language.
`event:portland-disturbance`	6 (0.1%)	Slug portland-disturbance is emitted by DISTURBANCE_CITY_PATTERNS when the named city appears near riot/unrest/protest/federal-response language.
`event:minneapolis-disturbance`	5 (0.0%)	Slug minneapolis-disturbance is emitted by DISTURBANCE_CITY_PATTERNS when the named city appears near riot/unrest/protest/federal-response language.

genre

Genre tags come from produced-video regexes, franchise parody detection, media-description sidecars, and one structural composite rule. The lineup rule is exact: reply + theme:criminal + exactly one media item.

scripts/tag_lexical.py:1055-1089

```
1055: PRODUCED_VIDEO_GENRE_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
1056:     ("genre:music-video", PATTERN_VIDEO_MUSIC_VIDEO),
1057:     ("genre:psa", PATTERN_VIDEO_PSA),
1058:     ("genre:recruitment", PATTERN_VIDEO_RECRUITMENT),
1059:     ("genre:advertisement", PATTERN_VIDEO_AD),
1060:     (
1061:         "genre:war-movie",
1062:         _compile(
1063:             r"\bwar[- ]movie\b|\bwar[- ]film\b|\baction[- ]movie\b"
1064:             r"|\b(?:cinematic|movie[- ]trailer|trailer[- ]style|dramatic)\b.{0,80}\b(?:war|battle|combat|military|raid|operation)\b"
1065:             r"|\b(?:war|battle|combat|military|raid|operation)\b.{0,80}\b(?:cinematic|movie[- ]trailer|trailer[- ]style|dramatic)\b"
1066:         ),
1067:     ),
1068:     (
1069:         "genre:utopian",
1070:         _compile(
1071:             r"\butopian\b|\bidealized\b|\baspirational\b|\b(?:golden\s+age|bright\s+future|morning\s+in\s+america)\b"
1072:             r"|\b(?:sunlit|heroic|triumphal)\b.{0,80}\b(?:montage|vision|future|aesthetic)\b"
1073:         ),
1074:     ),
1075:     (
1076:         "genre:dystopian",
1077:         _compile(
1078:             r"\bdystopian\b|\bsci[- ]?fi\b|\bscience[- ]fiction\b|\bcyberpunk\b"
1079:             r"|\b(?:dark|bleak|apocalyptic|hellscape|surveillance[- ]state|futuristic)\b.{0,80}\b(?:future|city|vision|scene|aesthetic)\b"
1080:         ),
1081:     ),
1082: )
1083: PRODUCED_VIDEO_STRUCTURE_TAGS = {
1084:     "video:produced",
1085:     "genre:music-video",
1086:     "video:montage",
1087:     "video:text-overlay",
1088:     "video:voiceover",
1089: }
```

scripts/tag_lexical.py:164-180

```
 164:     "Nepal",
 165:     "Netherlands",
 166:     "New Zealand",
 167:     "Nicaragua",
 168:     "Niger",
 169:     "Nigeria",
 170:     "Norway",
 171:     "Oman",
 172:     "Pakistan",
 173:     "Palau",
 174:     "Panama",
 175:     "Paraguay",
 176:     "Peru",
 177:     "Philippines",
 178:     "Poland",
 179:     "Portugal",
 180:     "Qatar",
```

scripts/tag_lexical.py:1709-1715

```
1709:     # genre:lineup: composite replies that hit theme:criminal with one photo.
1710:     if (
1711:         tweet_type == "reply"
1712:         and any(e["tag"] == "theme:criminal" for e in entries)
1713:         and media_count == 1
1714:     ):
```

```
1715:         add("genre:lineup")
```

scripts/tag_lexical.py:1730-1768

```
1730:     #   long   : > 120s
1731:     # We tag the bucket regardless of whether a kind matched so the viewer
1732:     # can filter "all videos under 30s" without depending on text cues.
1733:     if video_count > 0:
1734:         for pat, tag in (
1735:             (PATTERN_VIDEO_BODYCAM, "video:bodycam"),
1736:             (PATTERN_VIDEO_INTERVIEW, "video:interview"),
1737:             (PATTERN_VIDEO_NEWS_CLIP, "video:news-clip"),
1738:             (PATTERN_VIDEO_SPEECH, "video:speech"),
1739:         ):
1740:             m = pat.search(text)
1741:             if m:
1742:                 add(tag, span=m.span())
1743:         if PATTERN_AUDIO_MUSIC_CONTEXT.search(text):
1744:             add("audio:music-likely")
1745:         if PATTERN_AUDIO_MUSIC_CONTEXT.search(reply_context_text):
1746:             add("audio:music-likely", source="reply-context")
1747:         if any(e["tag"] in {"genre:music-video"} for e in entries):
1748:             add("audio:music-likely")
1749:         if m := PATTERN_PRODUCED_VIDEO_STYLE.search(text):
1750:             add("video:produced", span=m.span())
1751:         if video_max_duration_sec is not None and video_max_duration_sec > 0:
1752:             if video_max_duration_sec <= 30:
1753:                 add("video:short")
1754:             elif video_max_duration_sec <= 120:
1755:                 add("video:medium")
1756:             else:
1757:                 add("video:long")
1758:         # Derive genre:music-video ONLY from an explicit genre:music-video
1759:         # signal (set by the conservative describe_media / manual-review rules) —
1760:         # NEVER from audio:music-likely, which is an incidental acoustic heuristic.
1761:         # Also suppress it on speech / press-conference clips.
1762:         if not speech_indicator_present and any(e["tag"] == "genre:music-video" for e in entries):
1763:             add("genre:music-video")
1764:         if any(
1765:             e["tag"] in PRODUCED_VIDEO_STRUCTURE_TAGS or str(e["tag"]).startswith("genre:")
1766:             for e in entries
1767:         ):
1768:             add("video:produced")
```

Tag	Count	How it is produced
genre:lineup	670 (6.7%)	Composite rule: tweet_type == 'reply', theme:criminal already emitted, and media_count == 1.
genre:advertisement	439 (4.4%)	Emitted when PATTERN_VIDEO_AD or media sidecar genre matches the combined text buffer.
genre:psa	128 (1.3%)	Emitted when PATTERN_VIDEO_PSA or media sidecar genre matches the combined text buffer.
genre:recruitment	126 (1.3%)	Emitted when PATTERN_VIDEO_RECRUITMENT or media sidecar genre matches the combined text buffer.
genre:utopian	44 (0.4%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().
genre:music-video	36 (0.4%)	Emitted when PATTERN_VIDEO_MUSIC_VIDEO or media sidecar genre matches the combined text buffer.
genre:dystopian	18 (0.2%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().
genre:parody	5 (0.0%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().
genre:war-movie	2 (0.0%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().

legal

Legal tags are single-shot regexes over the combined text buffer for prosecution, civil litigation, or the explicit phrase birthright citizenship.

scripts/tag_lexical.py:814-826

```
 814: PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP = _compile(r"\bbirthright\s+citizenship\b")
 815:
 816: # Nativist birthright framing: "actual/real/true birthright of (every/all)
 817: # Americans", "American('s) birthright", "our birthright" + harm verb.
 818: # This is SEPARATE from bare "birthright citizenship" (which is just a policy
 819: # term) — we want nativism only when the framing posits Americans' heritage
 820: # entitlement as being stolen/diluted/erased/destroyed.
 821: PATTERN_THEME_NATIVISM_BIRTHRIGHT = _compile(
 822:     r"\b(?:actual|real|true)\s+birthright\s+of\s+(?:every|all)?\s*americans?\b"
 823:     r"|\bamerican(?:'?s?)?\s+birthright\b"
 824:     r"|\b(?:our|the)\s+birthright\b.{0,160}\b(?:steal|steals?|stolen|dilut|eras|destroy|destroyed|replac|betray|undermin)\b"
 825:     r"|\b(?:steal|steals?|stolen|dilut|eras|destroy|destroyed|replac|betray|undermin)\b.{0,160}\b(?:our|the)\s+birthright\b"
 826:     r"|\b(?:destroy(?:ing)?|destroys?|erasing?|replac(?:e|ing)?|betray(?:ing)?|undermin(?:e|ing)?|dilut(?:e|ing)?)"
```

scripts/tag_lexical.py:1178-1199

```
1178: PATTERN_LEGAL_CRIMINAL_PROSECUTION = _compile(
1179:     r"\bprosecut(?:e|ed|ing|ion|ions)\b"
1180:     r"|\bindict(?:ed|ment|ments)?\b"
1181:     r"|\bcharged\s+with\b"
1182:     r"|\bcriminal\s+(?:complaint|charges?|case|prosecution)\b"
1183:     r"|\barraign(?:ed|ment)?\b"
1184:     r"|\bple(?:a|d|aded|ads)\s+(?:guilty|not\s+guilty)\b"
1185:     r"|\bconvict(?:ed|ion|ions)?\b"
```

```
1186:     r"|\bsentenc(?:ed|ing)\b"
1187:     r"|\bfelony\s+charges?\b"
1188:     r"|\bmisdemeanor\s+charges?\b"
1189: )
1190: PATTERN_LEGAL_CIVIL_LAWSUIT = _compile(
1191:     r"\bcivil\s+(?:lawsuit|suit|action|case|complaint|litigation)\b"
1192:     r"|\blawsuits?\b"
1193:     r"|\bfiled\s+(?:a\s+)?(?:lawsuit|suit|civil\s+action|complaint)\b"
1194:     r"|\bsu(?:e|ed|es|ing)\b"
1195:     r"|\binjunction\b"
1196:     r"|\btemporary\s+restraining\s+order\b"
1197:     r"|\bTRO\b"
1198:     r"|\bsettlement\s+agreement\b"
1199:     r"|\bconsent\s+decree\b"
```

scripts/tag_lexical.py:1546-1563

```
1546:         (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
1547:         (PATTERN_LEGAL_CIVIL_LAWSUIT, "legal:civil-lawsuit"),
1548:         (PATTERN_TOPIC_ECONOMY, "topic:economy"),
1549:         (PATTERN_TOPIC_MILITARY, "topic:military"),
1550:         (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),
1551:         (PATTERN_EVENT_PALESTINE, "event:palestine"),
1552:         (PATTERN_THEME_BORDER, "policy:border"),
1553:         (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:         (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555:         (PATTERN_THEME_HOMELAND, "theme:homeland"),
1556:         (PATTERN_THEME_NATIVISM, "theme:nativism"),
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:         (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
1559:         (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560:         (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
1561:         (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
1562:         (PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP, "legal:birthright-citizenship"),
1563:         (PATTERN_THEME_NATIVISM_BIRTHRIGHT, "theme:nativism"),
```

Tag	Count	How it is produced
`legal:criminal-prosecution`	1,856 (18.5%)	Emitted when PATTERN_LEGAL_CRIMINAL_PROSECUTION matches the combined text buffer.
`legal:civil-lawsuit`	30 (0.3%)	Emitted when PATTERN_LEGAL_CIVIL_LAWSUIT matches the combined text buffer.
`legal:birthright-citizenship`	3 (0.0%)	Emitted when PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP matches the combined text buffer.

military

Military tags come from two sources: mentioned account aliases and regex matches for branches/service contexts. Any military:<branch> tag also implies topic:military.

scripts/tag_lexical.py:344-411

```
 344:     {
 345:         "ICEgov",
 346:         "CBP",
 347:         "USCIS",
 348:         "DHSgov",
 349:         "WhiteHouse",
 350:         "POTUS",
 351:         "PressSec",
 352:         "USDOL",
 353:         "RapidResponse47",
 354:         "HSI_HQ",
 355:         "USBPChief",
 356:         "SecMullinDHS",
 357:         "AGPamBondi",
 358:         "StateDept",
 359:         "DOJgov",
 360:         "FBI",
 361:         "DEAHQ",
 362:         "USMarshalsHQ",
 363:         "HHSGov",
 364:         "USTreasury",
 365:         "FEMA",
 366:         "DeptofWar",
 367:         "CENTCOM",
 368:         "Southcom",
 369:         "EPA",
 370:         "USCG",
 371:         "USCGAcademy",
 372:         "ComdtUSCG",
 373:         "VComdtUSCG",
 374:         "USCGLANTAREA",
 375:         "USCGPACAREA",
 376:         "USCGSoutheast",
 377:         "USCGHeartland",
 378:         "USCGNorCal",
 379:         "USCG_Tri_State",
 380:         "USArmyNorth",
 381:         "USNationalGuard",
 382:         "USNorthernCmd",
 383:         "TSA",
 384:         "SecretService",
 385:         "CISAgov",
 386:         "CISAInfraSec",
 387:         "CISACyber",
 388:         "FLETC",
 389:         "ODNIgov",
 390:         "NSAGov",
```

```
391:         "IPRCenter",
392:         "USAttyEssayli",
393:         "USAttyPirro",
394:         "ERO_LosAngeles",
395:         "ERO_HQ",
396:     }
397: )
398:
399: AGENCY_MENTION_ALIASES: dict[str, str] = {
400:     **{handle.lower(): handle for handle in AGENCY_HANDLES},
401:     "fbidirectorkash": "FBI",
402:     "fbimostwanted": "FBI",
403:     "fbimnneapolis": "FBI",
404:     "fbiminneapolis": "FBI",
405:     "fbitampa": "FBI",
406:     "thejusticedept": "DOJgov",
407:     "justiceoig": "DOJgov",
408:     "epaleezeldin": "EPA",
409:     "secwar": "DeptofWar",
410: }
411:
```

scripts/tag_lexical.py:568-619

```
568:     r"soldiers?|sailors?|airmen|marines|guardsmen|army|navy|air force|space force|"
569:     r"marine corps|coast guard|national guard|USAF|USSF|USMC|pentagon|"
570:     r"department of defense|DoD|DOD|veterans affairs|combat|battlefield|"
571:     r"war zone|deployed|deployment|carrier strike group|aircraft carrier|"
572:     r"carrier air wing|CVN\s*\d+|USS\s+[A-Z][A-Za-z0-9-]+|USNS\s+[A-Z][A-Za-z0-9-]+|"
573:     r"service academ(?:y|ies)|military academy|naval academy|coast guard academy|"
574:     r"air force academy|west point|USMA|USNA|USCGA|cadets?|midshipmen|"
575:     r"commander[- ]in[- ]chief|commandant)\b"
576: )
577: MILITARY_VOCAB: tuple[tuple[str, re.Pattern[str]], ...] = (
578:     ("army", _compile(r"\b(?:(?:u\.s\.\s+)?army|soldiers?|west\s+point|USMA)\b")),
579:     (
580:         "navy",
581:         _compile(
582:             r"\b(?:(?:u\.s\.\s+)?navy|sailors?|naval\s+academy|USNA|midshipmen|"
583:             r"carrier\s+strike\s+group|aircraft\s+carrier|carrier\s+air\s+wing|"
584:             r"CVN\s*\d+|USS\s+[A-Z][A-Za-z0-9-]+|USNS\s+[A-Z][A-Za-z0-9-]+)\b"
585:         ),
586:     ),
587:     (
588:         "air-force",
589:         _compile(r"\b(?:(?:u\.s\.\s+)?air\s+force|usaf|airm[ae]n|air\s+force\s+academy)\b"),
590:     ),
591:     ("space-force", _compile(r"\b(?:(?:u\.s\.\s+)?space\s+force|ussf)\b")),
592:     ("marines", _compile(r"\b(?:marine\s+corps|u\.s\.\s+marines?|usmc|marines)\b")),
593:     (
594:         "coast-guard",
595:         _compile(
596:             r"\b(?:coast\s+guard|USCG|USCGA|coast\s+guard\s+academy|coasties?|coast\s+guards?m[ae]n)\b"
597:         ),
598:     ),
599:     ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )
601: MILITARY_MENTION_ALIASES: dict[str, str] = {
602:     "usarmynorth": "army",
603:     "usnationalguard": "national-guard",
604:     "usguard": "national-guard",
605:     "usnavy": "navy",
606:     "usairforce": "air-force",
607:     "usspaceforce": "space-force",
608:     "usmc": "marines",
609:     "marines": "marines",
610:     "uscg": "coast-guard",
611:     "uscgacademy": "coast-guard",
612:     "comdtuscg": "coast-guard",
613:     "vcomdtuscg": "coast-guard",
614:     "uscglantarea": "coast-guard",
615:     "uscgpacarea": "coast-guard",
616:     "uscgsoutheast": "coast-guard",
617:     "uscgheartland": "coast-guard",
618:     "uscgnorcal": "coast-guard",
619:     "uscg_tri_state": "coast-guard",
```

scripts/tag_lexical.py:1631-1635

```
1631:     for slug, pat in MILITARY_VOCAB:
1632:         for m in pat.finditer(text):
1633:             add(f"military:{slug}", span=m.span())
1634:
1635:     if _general_topic_score(text) >= 3:
```

Tag	Count	How it is produced
`military:navy`	171 (1.7%)	Slug navy is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.
`military:coast-guard`	142 (1.4%)	Slug coast-guard is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.
`military:army`	43 (0.4%)	Slug army is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.
`military:national-guard`	38 (0.4%)	Slug national-guard is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.
`military:air-force`	37 (0.4%)	Slug air-force is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.

Tag	Count	How it is produced
`military:marines`	15 (0.1%)	Slug marines is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.
`military:space-force`	5 (0.0%)	Slug space-force is emitted from MILITARY_VOCAB text matches or account-mention aliases, and then implies topic:military.

origin

Origin tags are country extractions with stricter person-origin evidence: from <country>, or demonyms immediately followed by nouns like national, citizen, immigrant, migrant, man, woman, or descent.

scripts/tag_lexical.py:1217-1230

```
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!(?i:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," — anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?i:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?i:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" — anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
```

scripts/tag_lexical.py:1238-1316

```
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
```

```
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
```

scripts/tag_lexical.py:1677-1698

```
1677:     # origin:<COUNTRY> — validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> — broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
```

Tag	Count	How it is produced
origin:Mexico	378 (3.8%)	Slug Mexico is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:El-Salvador	94 (0.9%)	Slug El-Salvador is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Guatemala	91 (0.9%)	Slug Guatemala is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Honduras	86 (0.9%)	Slug Honduras is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Venezuela	54 (0.5%)	Slug Venezuela is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Cuba	52 (0.5%)	Slug Cuba is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Ecuador	25 (0.2%)	Slug Ecuador is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:China	23 (0.2%)	Slug China is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Iran	23 (0.2%)	Slug Iran is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Laos	23 (0.2%)	Slug Laos is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Colombia	22 (0.2%)	Slug Colombia is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Vietnam	18 (0.2%)	Slug Vietnam is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Haiti	16 (0.2%)	Slug Haiti is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Afghanistan	15 (0.1%)	Slug Afghanistan is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Dominican-Republic	14 (0.1%)	Slug Dominican-Republic is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Nicaragua	14 (0.1%)	Slug Nicaragua is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:India	13 (0.1%)	Slug India is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Jamaica	13 (0.1%)	Slug Jamaica is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Somalia	13 (0.1%)	Slug Somalia is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Brazil	12 (0.1%)	Slug Brazil is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.

Tag	Count	How it is produced
origin:Peru	6 (0.1%)	Slug Peru is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Guyana	5 (0.0%)	Slug Guyana is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Nigeria	5 (0.0%)	Slug Nigeria is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Russia	5 (0.0%)	Slug Russia is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Ukraine	5 (0.0%)	Slug Ukraine is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Cambodia	4 (0.0%)	Slug Cambodia is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Syria	4 (0.0%)	Slug Syria is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Kenya	3 (0.0%)	Slug Kenya is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Sierra-Leone	3 (0.0%)	Slug Sierra-Leone is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Sudan	3 (0.0%)	Slug Sudan is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Angola	2 (0.0%)	Slug Angola is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Barbados	2 (0.0%)	Slug Barbados is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Canada	2 (0.0%)	Slug Canada is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Chile	2 (0.0%)	Slug Chile is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Egypt	2 (0.0%)	Slug Egypt is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Ghana	2 (0.0%)	Slug Ghana is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Iraq	2 (0.0%)	Slug Iraq is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Pakistan	2 (0.0%)	Slug Pakistan is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Panama	2 (0.0%)	Slug Panama is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Philippines	2 (0.0%)	Slug Philippines is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Thailand	2 (0.0%)	Slug Thailand is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Turkey	2 (0.0%)	Slug Turkey is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Argentina	1 (0.0%)	Slug Argentina is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Azerbaijan	1 (0.0%)	Slug Azerbaijan is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Bangladesh	1 (0.0%)	Slug Bangladesh is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Burkina-Faso	1 (0.0%)	Slug Burkina-Faso is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Cameroon	1 (0.0%)	Slug Cameroon is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Costa-Rica	1 (0.0%)	Slug Costa-Rica is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Germany	1 (0.0%)	Slug Germany is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Ireland	1 (0.0%)	Slug Ireland is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Israel	1 (0.0%)	Slug Israel is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.

Tag	Count	How it is produced
origin:Italy	1 (0.0%)	Slug Italy is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Kazakhstan	1 (0.0%)	Slug Kazakhstan is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Latvia	1 (0.0%)	Slug Latvia is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Liberia	1 (0.0%)	Slug Liberia is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Mauritania	1 (0.0%)	Slug Mauritania is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Myanmar	1 (0.0%)	Slug Myanmar is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Rwanda	1 (0.0%)	Slug Rwanda is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:Senegal	1 (0.0%)	Slug Senegal is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.
origin:United-Kingdom	1 (0.0%)	Slug United-Kingdom is emitted when 'from <country>,' resolves to that country, or when a demonym for that country is followed by a person-origin noun.

parody

Parody tags come from a franchise gazetteer. A franchise match emits both genre:parody and parody:<franchise>; Star Wars also has a special two-cue rule for 'this is the way'.

scripts/tag_lexical.py:841-936

```
841: )
842:
843: # --- genre:parody + parody:<franchise> ----------------------------------
844: # A political/spoof parody of a recognizable media franchise.
845: # Franchise gazetteer: distinctive phrase -> franchise slug.
846: # "this is the way" alone is too generic (e.g. Mandalorian is pop-culture but
847: # appears in non-parody political text too), so we require at least ONE other
848: # Star Wars cue in the same text, OR the canonical "May the 4th/fourth be with
849: # you" phrase which is uniquely Star Wars in context.
850: PARODY_FRANCHISE_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (
851:     (
852:         _compile(
853:             r"\bmay\s+the\s+(?:4th|fourth)\s+be\s+with\s+you\b"
854:             r"|\bthe\s+force\s+(?:is|be|will\s+be)\s+with\b"
855:             r"|\b(?:jedi|sith|death\s+star|lightsaber|darth\s+vader|skywalker)\b"
856:             r"|\b(?:galaxy|republic|empire|rebellion|mandalorian)\b.{0,200}\b(?:jedi|sith|death\s+star|the\s+way)\b"
857:         ),
858:         "star-wars",
859:     ),
860:     (
861:         _compile(
862:             r"\bman\s+of\s+steel\b|\bkrypton(?:ite)?\b|\bfortress\s+of\s+solitude\b"
863:             r"|\bup,?\s+up,?\s+and\s+away\b|\bsuperman\b.{0,80}\b(?:cape|hero|krypton|steel|villain)\b"
864:         ),
865:         "superman",
866:     ),
867:     (
868:         _compile(
869:             r"\btop\s+gun\b|\bthe\s+need\s+for\s+speed\b|\bi\s+feel\s+the\s+need\b"
870:             r"|\bdanger\s+zone\b.{0,80}\b(?:maverick|jet|fly|top\s+gun)\b"
871:         ),
872:         "top-gun",
873:     ),
874:     (
875:         _compile(
876:             r"\bwinter\s+is\s+coming\b|\bgame\s+of\s+thrones\b|\bthe\s+iron\s+throne\b"
877:             r"|\byou\s+know\s+nothing,?\s+jon\s+snow\b"
878:         ),
879:         "game-of-thrones",
880:     ),
881:     (
882:         _compile(
883:             r"\bone\s+ring\s+to\s+rule\s+them\s+all\b|\byou\s+shall\s+not\s+pass\b"
884:             r"|\bmy\s+precious\b|\b(?:mordor|gandalf|frodo|the\s+shire)\b"
885:         ),
886:         "lord-of-the-rings",
887:     ),
888:     (
889:         _compile(
890:             r"\bhasta\s+la\s+vista,?\s+baby\b|\bskynet\b|\bthe\s+terminator\b"
891:             r"|\bi'?ll\s+be\s+back\b.{0,80}\b(?:terminator|machine|robot|cyborg)\b"
892:         ),
893:         "terminator",
894:     ),
895:     (
896:         _compile(r"\beye\s+of\s+the\s+tiger\b|\brocky\s+balboa\b|\bgonna\s+fly\s+now\b"),
897:         "rocky",
898:     ),
899:     (
900:         _compile(
901:             r"\bshaken,?\s+not\s+stirred\b|\blicen[sc]e\s+to\s+kill\b|\bagent\s+007\b|\bjames\s+bond\b"
```

```
902:         ),
903:         "james-bond",
904:     ),
905:     (
906:         _compile(
907:             r"\bavengers\s+assemble\b|\bwakanda\s+forever\b|\binfinity\s+(?:gauntlet|stones?)\b"
908:             r"|\bi\s+am\s+iron\s+man\b|\bthanos\b"
909:         ),
910:         "marvel",
911:     ),
912:     (
913:         _compile(r"\bwho\s+(?:you|ya)\s+gonna\s+call\b|\bghostbusters\b"),
914:         "ghostbusters",
915:     ),
916:     (
917:         _compile(r"\boffer\s+(?:he|you|they)\s+can'?t\s+refuse\b|\bthe\s+godfather\b"),
918:         "godfather",
919:     ),
920:     (
921:         _compile(r"\bpawn\s+stars\b"),
922:         "pawn-stars",
923:     ),
924: )
925: # Emit genre:parody ONLY when a franchise match fires; it never fires alone.
926: # A broader multi-cue Star Wars check: text must contain "this is the way"
927: # AND at least one other signature Star Wars cue.
928: PATTERN_PARODY_STAR_WARS_MULTI = _compile(r"\bthis\s+is\s+the\s+way\b")
929: PATTERN_PARODY_STAR_WARS_EXTRA_CUE = _compile(
930:     r"\b(?:may\s+the\s+(?:4th|fourth)\s+be\s+with\s+you|the\s+force|jedi|sith|death\s+star|"
931:     r"a\s+galaxy|galaxy\s+(?:far|that)|lightsaber|darth|yoda|skywalker|mandalorian)\b"
932: )
933:
934: # --- artist:<name> ----------------------------------------------------
935: # Named cultural figures / musicians and their signature songs. Identify
   ... [truncated in PDF; see source file for full pattern]
```

scripts/tag_lexical.py:1656-1670

```
1656:         if m := franchise_pat.search(text):
1657:             add("genre:parody", span=m.span())
1658:             add(f"parody:{franchise_slug}", span=m.span())
1659:             add("theme:pop-culture-reference")
1660:     # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661:     if (
1662:         PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:         and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:         and not any(e["tag"] == "parody:star-wars" for e in entries)
1665:     ):
1666:         m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:         if m_way:
1668:             add("genre:parody", span=m_way.span())
1669:             add("parody:star-wars", span=m_way.span())
1670:             add("theme:pop-culture-reference")
```

Tag	Count	How it is produced
`parody:star-wars`	2 (0.0%)	Slug star-wars is emitted by PARODY_FRANCHISE_GAZETTEER; the same match also emits genre:parody and theme:pop-culture-reference.
`parody:game-of-thrones`	1 (0.0%)	Slug game-of-thrones is emitted by PARODY_FRANCHISE_GAZETTEER; the same match also emits genre:parody and theme:pop-culture-reference.
`parody:ghostbusters`	1 (0.0%)	Slug ghostbusters is emitted by PARODY_FRANCHISE_GAZETTEER; the same match also emits genre:parody and theme:pop-culture-reference.
`parody:pawn-stars`	1 (0.0%)	Slug pawn-stars is emitted by PARODY_FRANCHISE_GAZETTEER; the same match also emits genre:parody and theme:pop-culture-reference.

phrase

Phrase tags are narrow literal domain terms. They are separate from broader slogans and imply topic:immigration.

scripts/tag_lexical.py:1176-1177

```
1176: PATTERN_PHRASE_MIGRANT = _compile(r"\bmigrants?\b")
1177: PATTERN_PHRASE_IMMIGRANT = _compile(r"\bimmigrants?\b")
```

scripts/tag_lexical.py:1586-1587

```
1586:         (PATTERN_PHRASE_MIGRANT, "phrase:migrant"),
1587:         (PATTERN_PHRASE_IMMIGRANT, "phrase:immigrant"),
```

scripts/tag_lexical.py:1883-1905

```
1883:     "slogan:criminal-illegal-alien": ("topic:immigration",),
1884:     "slogan:free-ticket-home": ("topic:immigration",),
1885:     "slogan:go-home": ("topic:immigration",),
1886:     "slogan:illegal-alien": ("topic:immigration",),
1887:     "slogan:mass-deportation": ("topic:immigration",),
1888:     "slogan:masa": ("topic:immigration",),
1889:     "slogan:find-and-kill": ("topic:immigration",),
1890:     "slogan:import-third-world": ("topic:immigration",),
1891:     "legal:birthright-citizenship": ("topic:immigration",),
1892:     "genre:parody": ("theme:pop-culture-reference",),
1893:     "slogan:most-secure-border": ("topic:immigration", "policy:border"),
1894:     "slogan:catch-release": ("topic:immigration",),
1895:     "slogan:project-homecoming": ("topic:immigration",),
1896:     "slogan:maga": ("topic:general",),
1897:     "slogan:maha": ("topic:general",),
1898:     "slogan:america-first": ("topic:general",),
```

```
1899:     "slogan:golden-age": ("topic:general",),
1900:     "slogan:save-america": ("topic:general",),
1901:     "slogan:law-and-order": ("topic:general",),
1902:     "slogan:peace-through-strength": ("topic:general",),
1903:     "slogan:promises-kept": ("topic:general",),
1904:     "phrase:immigrant": ("topic:immigration",),
1905:     "phrase:migrant": ("topic:immigration",),
```

Tag	Count	How it is produced
`phrase:immigrant`	234 (2.3%)	Config pattern in config/tag_taxonomy.yaml: \bimmigrants?\b
`phrase:migrant`	142 (1.4%)	Config pattern in config/tag_taxonomy.yaml: \bmigrants?\b

policy

Policy tags are regexes for specific policy/program areas, not broad topics: border, sanctuary, worksite enforcement, and CBP Home/self-deportation campaign language.

scripts/tag_lexical.py:652-656

```
 652: PATTERN_THEME_BORDER = _compile(r"\b(border|southwest border|crossing|border wall)\b")
 653: PATTERN_THEME_SANCTUARY = _compile(
 654:     r"\b(sanctuary (?:cit(?:y|ies)|jurisdiction|polic(?:y|ies))|sanctuary state)\b"
 655: )
 656: PATTERN_THEME_WORKSITE = _compile(r"\b(worksite|workplace|I-9|E-Verify|employer (?:audit|raid))\b")
```

scripts/tag_lexical.py:786-797

```
 786: PATTERN_THEME_CBP_HOME = _compile(
 787:     r"\bCBP\s+Home\b"
 788:     r"|\bProject\s+Homecoming\b"
 789:     r"|\bself[- ]deport(?:ed|ing|ation)?\b"
 790:     r"|\bfree\s+(?:flight|plane\s+ticket|ticket)\s+home\b"
 791:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
 792:     r"|\bexit\s+bonus\b"
 793:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
 794:     r"|\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
 795:     r"[\s\S]{0,120}\bgo\s+home\b"
 796:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
 797:     r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
```

scripts/tag_lexical.py:1552-1560

```
1552:         (PATTERN_THEME_BORDER, "policy:border"),
1553:         (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:         (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555:         (PATTERN_THEME_HOMELAND, "theme:homeland"),
1556:         (PATTERN_THEME_NATIVISM, "theme:nativism"),
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:         (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
1559:         (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560:         (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
```

Tag	Count	How it is produced
`policy:border`	1,071 (10.7%)	Config pattern in config/tag_taxonomy.yaml: \b(border|southwest border|crossing|border wall)\b
`policy:cbp-home`	236 (2.4%)	Emitted when PATTERN_THEME_CBP_HOME matches the combined text buffer.
`policy:sanctuary-cities`	191 (1.9%)	Config pattern in config/tag_taxonomy.yaml: \b(sanctuary (?:cit(?:y|ies)|jurisdiction|polic(?:y|ies))|sanctuary state)\b
`policy:worksite-enforcement`	21 (0.2%)	Config pattern in config/tag_taxonomy.yaml: \b(worksite|workplace|I-9|E-Verify|employer (?:audit|raid))\b

religion

Religion tags are regex-based. Christianity is a direct Christian-specific regex. The broader theme:religion rule explicitly suppresses common expletive uses nearby.

scripts/tag_lexical.py:681-709

```
 681: PATTERN_THEME_CHRISTIANITY = _compile(
 682:     r"\bchristian(?:ity|s)?\b"
 683:     r"|\bjudeo[- ]christian\b"
 684:     r"|\bchristian\s+(?:faith|values?|church|churches|heritage|nation)\b"
 685:     r"|\bjesus(?:\s+christ)?\b"
 686:     r"|\bchrist\s+(?:is\s+king|the\s+king|our\s+lord)\b"
 687:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
 688:     r"|\b(?:[1-3]\s+)?(?:Genesis|Exodus|Leviticus|Numbers|Deuteronomy|Joshua|Judges|Ruth|"
 689:     r"Samuel|Kings|Chronicles|Ezra|Nehemiah|Esther|Job|Psalms?|Proverbs|Ecclesiastes|"
 690:     r"Song\s+of\s+Songs|Isaiah|Jeremiah|Lamentations|Ezekiel|Daniel|Hosea|Joel|Amos|"
 691:     r"Obadiah|Jonah|Micah|Nahum|Habakkuk|Zephaniah|Haggai|Zechariah|Malachi|Matthew|"
 692:     r"Mark|Luke|John|Acts|Romans|Corinthians|Galatians|Ephesians|Philippians|"
 693:     r"Colossians|Thessalonians|Timothy|Titus|Philemon|Hebrews|James|Peter|Jude|"
 694:     r"Revelation)\s+\d{1,3}:\d{1,3}(?:[-\u2013]\d{1,3})?\b"
 695:     r"|\bin\s+jesus(?:'s|')?\s+name\b"
 696: )
 697: PATTERN_THEME_RELIGION = _compile(
 698:     r"\breligio(?:n|us)\b"
 699:     r"|\bfaith(?:ful|s|[- ]based)?\b"
 700:     r"|\b(?:worship|church(?:es)?|synagogue|mosque|temple|chaplain)\b"
 701:     r"|\b(?:prayer|prayers|pray|praying|prayed)\b"
 702:     r"|\b(?:bless|blessed|blessing|blessings)\b"
```

```
 703:     r"|\b(?:god|lord|jesus|christ|christian(?:ity|s)?|judeo[- ]christian)\b"
 704:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
 705: )
 706: PATTERN_THEME_RELIGION_EXPLETIVE = _compile(
 707:     r"\b(?:oh\s+my\s+god|omg|god\s*damn(?:ed|it)?|goddamn(?:ed)?|"
 708:     r"for\s+god'?s\s+sake|good\s+lord|thank\s+god)\b"
 709: )
```

scripts/tag_lexical.py:1557-1557

```
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
```

scripts/tag_lexical.py:1777-1787

```
1777:     for match in PATTERN_THEME_RELIGION.finditer(text):
1778:         start, end = match.span()
1779:         window = text[
1780:             max(0, start - RELIGION_EXPLETIVE_WINDOW_CHARS) : min(
1781:                 len(text), end + RELIGION_EXPLETIVE_WINDOW_CHARS
1782:             )
1783:         ]
1784:         if PATTERN_THEME_RELIGION_EXPLETIVE.search(window):
1785:             continue
1786:         return match
1787:     return None
```

Tag	Count	How it is produced
`religion:christianity`	85 (0.8%)	Emitted when PATTERN_THEME_CHRISTIANITY matches the combined text buffer.

slogan

Slogan tags are regexes for recurring stock phrases and campaign/administration branding. A slogan match can also imply broader topic/policy tags through the parent-topic map.

scripts/tag_lexical.py:832-839

```
 832: PATTERN_SLOGAN_FIND_AND_KILL = _compile(
 833:     r"\bwe\s+will\s+find\s+you\s+(?:and|&)\s+(?:we\s+will\s+)?kill\s+you\b"
 834:     r"|\bfind\s+you\s+(?:and|&)\s+kill\s+you\b"
 835: )
 836:
 837: # --- slogan:import-third-world ------------------------------------------
 838: PATTERN_SLOGAN_IMPORT_THIRD_WORLD = _compile(
 839:     r"\bimport\s+the\s+third\s+world\b"
```

scripts/tag_lexical.py:1145-1175

```
1145: PATTERN_SLOGAN_NICE = _compile(r"\b(NICE day|NICE morning|ICE is NICE|NICE city)\b")
1146: PATTERN_SLOGAN_WORST = _compile(r"\bWORST OF THE WORST\b")
1147: PATTERN_SLOGAN_REPORTRECON = _compile(r"\bReport\.\s*Recon\.\s*Raid\.")
1148: PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN = _compile(r"\bcriminal\s+illegal\s+aliens?\b")
1149: PATTERN_SLOGAN_ILLEGAL_ALIEN = _compile(r"\billegal\s+aliens?\b")
1150: PATTERN_SLOGAN_FREE_TICKET_HOME = _compile(
1151:     r"\bfree\s+(?:ticket|flight|plane\s+ticket)\s+home\b"
1152:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
1153:     r"|\bfree\s+flight\s+to\s+(?:your|their|his|her)\s+home\s+country\b"
1154: )
1155: PATTERN_SLOGAN_GO_HOME = _compile(
1156:     r"\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
1157:     r"[\s\S]{0,120}\bgo\s+home\b"
1158:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
1159:     r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
1160: )
1161: PATTERN_SLOGAN_PROJECT_HOMECOMING = _compile(r"\bProject\s+Homecoming\b")
1162: PATTERN_SLOGAN_MAGA = _compile(r"\bMAGA\b|\bMake\s+America\s+Great\s+Again\b")
1163: PATTERN_SLOGAN_MAHA = _compile(r"\bMAHA\b|\bMake\s+America\s+Healthy\s+Again\b")
1164: PATTERN_SLOGAN_MASA = _compile(r"\bMake\s+America\s+Safe\s+Again\b")
1165: PATTERN_SLOGAN_AMERICA_FIRST = _compile(r"\bAmerica\s+First\b|\bAmericaFirst\b")
1166: PATTERN_SLOGAN_GOLDEN_AGE = _compile(r"\bGolden\s+Age\b|\bWelcome\s+to\s+the\s+Golden\s+Age\b")
1167: PATTERN_SLOGAN_SAVE_AMERICA = _compile(r"\bSave\s+America\b|\bSAVEAMERICA\b")
1168: PATTERN_SLOGAN_LAW_AND_ORDER = _compile(r"\bLaw\s+and\s+Order\b|\bLaw\s*&\s*Order\b")
1169: PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH = _compile(r"\bPeace\s+Through\s+Strength\b")
1170: PATTERN_SLOGAN_PROMISES_KEPT = _compile(
1171:     r"\bPromises?\s+Made[,;:]?\s+Promises?\s+Kept\b|\bPromises?\s+Kept\b"
1172: )
1173: PATTERN_SLOGAN_MASS_DEPORTATION = _compile(r"\bMass\s+Deportations?\b")
1174: PATTERN_SLOGAN_MOST_SECURE_BORDER = _compile(r"\bmost\s+secure(?:d)?\s+border\b")
1175: PATTERN_SLOGAN_CATCH_RELEASE = _compile(r"\bcatch(?:-and-|\s+and\s+)release\b")
```

scripts/tag_lexical.py:1564-1585

```
1564:         (PATTERN_SLOGAN_NICE, "slogan:nice"),
1565:         (PATTERN_SLOGAN_WORST, "slogan:worst"),
1566:         (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
1567:         (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
1568:         (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
1569:         (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
1570:         (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
1571:         (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homecoming"),
1572:         (PATTERN_SLOGAN_MAGA, "slogan:maga"),
1573:         (PATTERN_SLOGAN_MAHA, "slogan:maha"),
1574:         (PATTERN_SLOGAN_MASA, "slogan:masa"),
1575:         (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
1576:         (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
1577:         (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
1578:         (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
1579:         (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
1580:         (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
1581:         (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
1582:         (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
```

```
1583:         (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
1584:         (PATTERN_SLOGAN_FIND_AND_KILL, "slogan:find-and-kill"),
1585:         (PATTERN_SLOGAN_IMPORT_THIRD_WORLD, "slogan:import-third-world"),
```

Tag	Count	How it is produced
slogan:illegal-alien	2,287 (22.9%)	Config pattern in config/tag_taxonomy.yaml: \billegal\s+aliens?\b
slogan:criminal-illegal-alien	1,402 (14.0%)	Config pattern in config/tag_taxonomy.yaml: \bcriminal\s+illegal\s+aliens?\b
slogan:worst	222 (2.2%)	Config pattern in config/tag_taxonomy.yaml: \bWORST OF THE WORST\b
slogan:masa	140 (1.4%)	Emitted when PATTERN_SLOGAN_MASA matches the combined text buffer.
slogan:america-first	72 (0.7%)	Emitted when PATTERN_SLOGAN_AMERICA_FIRST matches the combined text buffer.
slogan:maga	58 (0.6%)	Emitted when PATTERN_SLOGAN_MAGA matches the combined text buffer.
slogan:law-and-order	51 (0.5%)	Emitted when PATTERN_SLOGAN_LAW_AND_ORDER matches the combined text buffer.
slogan:promises-kept	50 (0.5%)	Emitted when PATTERN_SLOGAN_PROMISES_KEPT matches the combined text buffer.
slogan:most-secure-border	45 (0.4%)	Emitted when PATTERN_SLOGAN_MOST_SECURE_BORDER matches the combined text buffer.
slogan:golden-age	42 (0.4%)	Emitted when PATTERN_SLOGAN_GOLDEN_AGE matches the combined text buffer.
slogan:free-ticket-home	26 (0.3%)	Emitted when PATTERN_SLOGAN_FREE_TICKET_HOME matches the combined text buffer.
slogan:mass-deportation	26 (0.3%)	Emitted when PATTERN_SLOGAN_MASS_DEPORTATION matches the combined text buffer.
slogan:save-america	24 (0.2%)	Emitted when PATTERN_SLOGAN_SAVE_AMERICA matches the combined text buffer.
slogan:go-home	16 (0.2%)	Emitted when PATTERN_SLOGAN_GO_HOME matches the combined text buffer.
slogan:peace-through-strength	12 (0.1%)	Emitted when PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH matches the combined text buffer.
slogan:catch-release	11 (0.1%)	Emitted when PATTERN_SLOGAN_CATCH_RELEASE matches the combined text buffer.
slogan:import-third-world	11 (0.1%)	Emitted when PATTERN_SLOGAN_IMPORT_THIRD_WORLD matches the combined text buffer.
slogan:maha	11 (0.1%)	Emitted when PATTERN_SLOGAN_MAHA matches the combined text buffer.
slogan:nice	9 (0.1%)	Config pattern in config/tag_taxonomy.yaml: \b(NICE day|NICE morning|ICE is NICE|NICE city)\b
slogan:project-homecoming	4 (0.0%)	Emitted when PATTERN_SLOGAN_PROJECT_HOMECOMING matches the combined text buffer.
slogan:find-and-kill	2 (0.0%)	Emitted when PATTERN_SLOGAN_FIND_AND_KILL matches the combined text buffer.
slogan:reportrecon	2 (0.0%)	Config pattern in config/tag_taxonomy.yaml: \bReport\.\s*Recon\.\s*Raid\.

speaker

Speaker tags are emitted only when a known speaker alias occurs near speech/interview/remarks/announcement context. The code does not infer speakers from faces or setting.

scripts/tag_lexical.py:1106-1139

```
1106:     r"say|says|said|speak(?:s|ing)?|spoke|remark(?:s|ed)?|brief(?:s|ed|ing)?|"
1107:     r"join(?:s|ed|ing)?|interview(?:s|ed|ing)?|sat\s+down|quote(?:s|d)?)"
1108: )
1109: SPEAKER_NOUN_CONTEXT = (
1110:     r"(?:remarks?|speech|address|statement|announcement|interview|quote|"
1111:     r"press\s+(?:conference|briefing|gaggle)|briefing)"
1112: )
1113: SPEAKER_ALIASES: tuple[tuple[str, str], ...] = (
1114:     (
1115:         "First Lady Melania Trump",
1116:         r"(?:First\s+Lady\s+)?Melania\s+Trump|FLOTUS",
1117:     ),
1118:     (
1119:         "Secretary Mullin",
1120:         r"Secretary\s+Mullin|Sec\.?\s+Mullin|@?SecMullinDHS",
1121:     ),
1122:     (
1123:         "President Trump",
1124:         r"President\s+(?:Donald\s+J\.?\s+)?Trump|Donald\s+Trump|@?POTUS|@?realDonaldTrump",
1125:     ),
1126:     (
1127:         "Vice President Vance",
1128:         r"Vice\s+President\s+(?:JD\s+|J\.D\.\s+)?Vance|VP\s+Vance|J\.?D\.?\s+Vance|@?VP",
1129:     ),
1130:     (
1131:         "Tom Homan",
1132:         r"Tom\s+Homan|Thomas\s+D\.?\s+Homan|@?RealTomHoman",
1133:     ),
1134:     (
1135:         "Stephen Miller",
1136:         r"Stephen\s+Miller|@?StephenM",
1137:     ),
1138:     (
1139:         "Gregory Bovino",
```

scripts/tag_lexical.py:2202-2217

```
2202:     """Return speaker tags only when a named official is tied to speech."""
```

```
    out: list[tuple[str, tuple[int, int]]] = []
    for canonical, alias in SPEAKER_ALIASES:
        patterns = (
            rf"(?<!\w)({alias})(?!\w).{{0,100}}\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\b",
            rf"\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\b"
            rf".{{0,80}}\b(?:by|from|with|of|featuring|:)?\s*({alias})(?!\w)",
        )
        for pattern in patterns:
            match = re.search(pattern, text, re.I)
            if match:
                out.append((f"speaker:{canonical}", match.span(1)))
                break
    return out
```

Tag	Count	How it is produced
`speaker:President Trump`	392 (3.9%)	Slug President Trump is emitted only when a speaker alias appears near speech, remarks, announcement, interview, quote, or briefing context.
`speaker:Vice President Vance`	31 (0.3%)	Slug Vice President Vance is emitted only when a speaker alias appears near speech, remarks, announcement, interview, quote, or briefing context.
`speaker:Secretary Mullin`	22 (0.2%)	Slug Secretary Mullin is emitted only when a speaker alias appears near speech, remarks, announcement, interview, quote, or briefing context.
`speaker:Tom Homan`	11 (0.1%)	Slug Tom Homan is emitted only when a speaker alias appears near speech, remarks, announcement, interview, quote, or briefing context.
`speaker:First Lady Melania Trump`	6 (0.1%)	Slug First Lady Melania Trump is emitted only when a speaker alias appears near speech, remarks, announcement, interview, quote, or briefing context.
`speaker:Stephen Miller`	3 (0.0%)	Slug Stephen Miller is emitted only when a speaker alias appears near speech, remarks, announcement, interview, quote, or briefing context.

state

State tags are validated extractions from the '<Place>, <State>' pattern, resolved against the 50-state vocabulary and normalized into the slug.

scripts/tag_lexical.py:233-287

```
US_STATES: tuple[str, ...] = (
    "Alabama",
    "Alaska",
    "Arizona",
    "Arkansas",
    "California",
    "Colorado",
    "Connecticut",
    "Delaware",
    "Florida",
    "Georgia",
    "Hawaii",
    "Idaho",
    "Illinois",
    "Indiana",
    "Iowa",
    "Kansas",
    "Kentucky",
    "Louisiana",
    "Maine",
    "Maryland",
    "Massachusetts",
    "Michigan",
    "Minnesota",
    "Mississippi",
    "Missouri",
    "Montana",
    "Nebraska",
    "Nevada",
    "New Hampshire",
    "New Jersey",
    "New Mexico",
    "New York",
    "North Carolina",
    "North Dakota",
    "Ohio",
    "Oklahoma",
    "Oregon",
    "Pennsylvania",
    "Rhode Island",
    "South Carolina",
    "South Dakota",
    "Tennessee",
    "Texas",
    "Utah",
    "Vermont",
    "Virginia",
    "Washington",
    "West Virginia",
    "Wisconsin",
    "Wyoming",
)
STATE_BY_LOWER: dict[str, str] = {s.lower(): s for s in US_STATES}
```

scripts/tag_lexical.py:1229-1230

```
1229: # "<Place>, <State>" — anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
```

scripts/tag_lexical.py:1703-1708

```
1703:     # state:<NAME> — the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
```

scripts/tag_lexical.py:2046-2051

```
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

Tag	Count	How it is produced
`state:Texas`	133 (1.3%)	Slug Texas is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:California`	89 (0.9%)	Slug California is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:New-York`	57 (0.6%)	Slug New-York is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Florida`	56 (0.6%)	Slug Florida is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Virginia`	46 (0.5%)	Slug Virginia is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:North-Carolina`	23 (0.2%)	Slug North-Carolina is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Utah`	22 (0.2%)	Slug Utah is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Illinois`	21 (0.2%)	Slug Illinois is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Tennessee`	16 (0.2%)	Slug Tennessee is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Minnesota`	15 (0.1%)	Slug Minnesota is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Washington`	15 (0.1%)	Slug Washington is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:New-Jersey`	14 (0.1%)	Slug New-Jersey is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Colorado`	13 (0.1%)	Slug Colorado is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Oregon`	13 (0.1%)	Slug Oregon is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Pennsylvania`	13 (0.1%)	Slug Pennsylvania is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Arizona`	12 (0.1%)	Slug Arizona is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Maryland`	12 (0.1%)	Slug Maryland is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Massachusetts`	12 (0.1%)	Slug Massachusetts is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Connecticut`	9 (0.1%)	Slug Connecticut is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Georgia`	9 (0.1%)	Slug Georgia is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Louisiana`	7 (0.1%)	Slug Louisiana is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Michigan`	7 (0.1%)	Slug Michigan is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:South-Carolina`	7 (0.1%)	Slug South-Carolina is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
`state:Missouri`	6 (0.1%)	Slug Missouri is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.

Tag	Count	How it is produced
state:Nevada	6 (0.1%)	Slug Nevada is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:New-Hampshire	6 (0.1%)	Slug New-Hampshire is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Wisconsin	6 (0.1%)	Slug Wisconsin is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Alabama	5 (0.0%)	Slug Alabama is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Idaho	5 (0.0%)	Slug Idaho is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Iowa	5 (0.0%)	Slug Iowa is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Vermont	5 (0.0%)	Slug Vermont is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Indiana	4 (0.0%)	Slug Indiana is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Kansas	4 (0.0%)	Slug Kansas is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Mississippi	4 (0.0%)	Slug Mississippi is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Kentucky	3 (0.0%)	Slug Kentucky is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:New-Mexico	3 (0.0%)	Slug New-Mexico is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Oklahoma	3 (0.0%)	Slug Oklahoma is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Hawaii	2 (0.0%)	Slug Hawaii is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Ohio	2 (0.0%)	Slug Ohio is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Alaska	1 (0.0%)	Slug Alaska is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Arkansas	1 (0.0%)	Slug Arkansas is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Delaware	1 (0.0%)	Slug Delaware is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Maine	1 (0.0%)	Slug Maine is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:Nebraska	1 (0.0%)	Slug Nebraska is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:North-Dakota	1 (0.0%)	Slug North-Dakota is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.
state:South-Dakota	1 (0.0%)	Slug South-Dakota is emitted only when the '<Place>, <State>' regex captures a state name and resolves it against the U.S. state vocabulary.

subject

Subject tags are a mix of direct regexes and deterministic composites. subject:enforcement-op is not a visual guess here; it is emitted when both action:detention and theme:criminal have fired.

scripts/tag_lexical.py:780-804

```
780: PATTERN_SUBJECT_CBP_HOME_APP = _compile(
781:     r"\b@?CBP\s+Home\s+App\b"
782:     r"|\b@?CBP\s+Home\b"
783:     r"|\bDHS\.?GOV/CBPHOME\b"
784:     r"|\b(?:dhs|cbp)\.gov/(?:cbp[-_]?home|projecthomecoming)\b"
785: )
786: PATTERN_THEME_CBP_HOME = _compile(
787:     r"\bCBP\s+Home\b"
788:     r"|\bProject\s+Homecoming\b"
789:     r"|\bself[- ]deport(?:ed|ing|ation)?\b"
790:     r"|\bfree\s+(?:flight|plane\s+ticket|ticket)\s+home\b"
791:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
792:     r"|\bexit\s+bonus\b"
793:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
794:     r"|\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
795:     r"[\s\S]{0,120}\bgo\s+home\b"
796:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
797:     r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
```

```
 798: )
 799: PATTERN_SUBJECT_CELEBRITY = _compile(
 800:     r"\bSydney\s+Sweeney\b"
 801:     r"|\b(?:actress|influencer|celebrity|pop\s+star|movie\s+star|Hollywood\s+star)"
 802:     r"\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+\b"
 803:     r"|\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+,\s+(?:an?\s+)?"
 804:     r"(?:actress|influencer|celebrity|pop\s+star|movie\s+star)\b"
```

scripts/tag_lexical.py:1209-1216

```
1209: PATTERN_ANGEL_FAMILY = _compile(
1210:     r"\bangel (?:famil(?:y|ies)|mom|dad|parent|mother|father|wife|husband|son|daughter|child)\b"
1211: )
1212: PATTERN_NATIVE_BORN_CITIZEN = _compile(
1213:     r"\bnative[- ]born\s+(?:citizens?|americans?|u\.?s\.?\s+citizens?|people|workers|taxpayers)\b"
1214: )
1215: # Place names are captured case-insensitively (so "from mexico" is caught,
1216: # not just "from Mexico") after a small set of prepositions. The optional
```

scripts/tag_lexical.py:1717-1725

```
1717:     # subject:enforcement-op heuristic from the deterministic rules above:
1718:     # if both action:detention and theme:criminal fire, that's strong enough
1719:     # to set this without tentative.
1720:     if any(e["tag"] == "action:detention" for e in entries) and any(
1721:         e["tag"] == "theme:criminal" for e in entries
1722:     ):
1723:         add("subject:enforcement-op")
1724:
1725:     # video:<kind> + video:duration-bucket — only when a video is attached.
```

Tag	Count	How it is produced
subject:enforcement-op	1,351 (13.5%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().
subject:cbp-home-app	189 (1.9%)	Emitted when PATTERN_SUBJECT_CBP_HOME_APP matches the combined text buffer.
subject:angel-family	54 (0.5%)	Emitted when PATTERN_ANGEL_FAMILY matches the combined text buffer.
subject:native-born-citizen	3 (0.0%)	Emitted when PATTERN_NATIVE_BORN_CITIZEN matches the combined text buffer.
subject:celebrity	2 (0.0%)	Emitted when PATTERN_SUBJECT_CELEBRITY matches the combined text buffer.
subject:detainee	1 (0.0%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().
subject:official	1 (0.0%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().

theme

Theme tags are rhetorical-frame regexes plus a few guarded helper functions, such as religion expletive suppression, coded nativism context, and martyrdom context windows.

scripts/tag_lexical.py:657-725

```
 657: PATTERN_THEME_HOMELAND = re.compile(
 658:     r"\b(?i:our|the|this|america's)\s+Homeland\b"
 659:     r"|\b(?i:protect|protecting|secure|securing|defend|defending|safeguard|safeguarding)"
 660:     r"\s+(?:(?i:our|the|this|america's)\s+)?Homeland\b"
 661:     r"|\b(?i:safe|secure)\s+Homeland\b"
 662: )
 663: PATTERN_THEME_NATIVISM = _compile(
 664:     r"\bnative[- ]born\s+(?:americans?|workers?|citizens?)\b"
 665:     r"|\bamerican[- ]born\s+(?:americans?|workers?|citizens?)\b"
 666:     r"|\bforeign[- ]born\s+workers?\b"
 667:     r"|\bforeign\s+(?:workers?|labor)\b.{0,140}\b(?:flood|cheap|displac|replac|"
 668:     r"betray|job market|american\s+(?:workers?|jobs?)|americans?\s+first)\b"
 669:     r"|\b(?:american\s+(?:workers?|jobs?)|americans?\s+first|job market)\b.{0,140}"
 670:     r"\b(?:foreign\s+(?:workers?|labor)|foreign[- ]born\s+workers?)\b"
 671:     r"|\bglobalism has failed\b|\bamericanism will prevail\b"
 672:     r"|\b(?:our\s+)?forefathers?\b"
 673: )
 674: PATTERN_NATIVISM_INHERITANCE = _compile(r"\b(?:inheritors?|inheritance|inherit(?:ed|ing)?)\b")
 675: PATTERN_NATIVISM_INHERITANCE_CONTEXT = _compile(
 676:     r"\b(?:american|america|nation|national|citizens?|native[- ]born|foreign[- ]born|"
 677:     r"immigrants?|migrants?|aliens?|homeland|heritage|birthright|forefathers?|"
 678:     r"founders?|ancestors?|descendants?|civilization|our\s+people|our\s+country|"
 679:     r"our\s+nation|american\s+(?:workers?|jobs?))\b"
 680: )
 681: PATTERN_THEME_CHRISTIANITY = _compile(
 682:     r"\bchristian(?:ity|s)?\b"
 683:     r"|\bjudeo[- ]christian\b"
 684:     r"|\bchristian\s+(?:faith|values?|church|churches|heritage|nation)\b"
 685:     r"|\bjesus(?:\s+christ)?\b"
 686:     r"|\bchrist\s+(?:is\s+king|the\s+king|our\s+lord)\b"
 687:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
 688:     r"|\b(?:[1-3]\s+)?(?:Genesis|Exodus|Leviticus|Numbers|Deuteronomy|Joshua|Judges|Ruth|"
 689:     r"Samuel|Kings|Chronicles|Ezra|Nehemiah|Esther|Job|Psalms?|Proverbs|Ecclesiastes|"
 690:     r"Song\s+of\s+Songs|Isaiah|Jeremiah|Lamentations|Ezekiel|Daniel|Hosea|Joel|Amos|"
 691:     r"Obadiah|Jonah|Micah|Nahum|Habakkuk|Zephaniah|Haggai|Zechariah|Malachi|Matthew|"
 692:     r"Mark|Luke|John|Acts|Romans|Corinthians|Galatians|Ephesians|Philippians|"
 693:     r"Colossians|Thessalonians|Timothy|Titus|Philemon|Hebrews|James|Peter|Jude|"
 694:     r"Revelation)\s+\d{1,3}:\d{1,3}(?:[-\u2013]\d{1,3})?\b"
 695:     r"|\bin\s+jesus(?:'s|')?\s+name\b"
 696: )
 697: PATTERN_THEME_RELIGION = _compile(
 698:     r"\breligio(?:n|us)\b"
```

```
    r"|\bfaith(?:ful|s|[- ]based)?\b"
    r"|\b(?:worship|church(?:es)?|synagogue|mosque|temple|chaplain)\b"
    r"|\b(?:prayer|prayers|pray|praying|prayed)\b"
    r"|\b(?:bless|blessed|blessing|blessings)\b"
    r"|\b(?:god|lord|jesus|christ|christian(?:ity|s)?|judeo[- ]christian)\b"
    r"|\b(?:bible|biblical|scripture|scriptural)\b"
)
PATTERN_THEME_RELIGION_EXPLETIVE = _compile(
    r"\b(?:oh\s+my\s+god|omg|god\s*damn(?:ed|it)?|goddamn(?:ed)?|"
    r"for\s+god'?s\s+sake|good\s+lord|thank\s+god)\b"
)
RELIGION_EXPLETIVE_WINDOW_CHARS = 18
PATTERN_THEME_TRANSGENDER = _compile(
    r"\btransgender\b"
    r"|\bgender[- ](?:ideology|identity|affirming|transition|dysphoria)\b"
    r"|\b(?:biological|transgender)\s+"
    r"(?:males?|females?|man|woman|men|women|boys?|girls?)\b"
    r"|\b(?:men|males|boys)\s+in\s+(?:women[\u2019']?s|girls[\u2019']?)\s+sports\b"
    r"|\b(?:men|males|boys)\s+(?:competing|playing|participating)\s+"
    r"(?:in|against|with)\s+(?:women|girls|female\s+athletes?)\b"
    r"|\b(?:women[\u2019']?s|girls[\u2019']?)\s+sports\s+(?:are\s+)?"
    r"for\s+(?:women|girls|female\s+athletes?)\b"
    r"|\b(?:protect|save|defend)\s+(?:women[\u2019']?s|girls[\u2019']?)\s+sports\b"
    r"|\btitle\s+ix\b.{0,80}\b(?:transgender|gender|women[\u2019']?s\s+sports|girls[\u2019']?\s+sports)\b"
    r"|\b(?:transgender|gender|women[\u2019']?s\s+sports|girls[\u2019']?\s+spo
   ... [truncated in PDF; see source file for full pattern]
```

scripts/tag_lexical.py:765-779

```
PATTERN_MARTYRDOM_WHY = _compile(
    r"\bthis\s+is\s+our\s+why\b"
    r"|\bthey\s+are\s+our\s+why\b"
    r"|\byou\s+are\s+why\s+we\s+fight\b"
)
PATTERN_MARTYRDOM_CONTEXT = _compile(
    r"\bangel\s+(?:famil(?:y|ies)|mom|dad|parent|mother|father|wife|husband|son|daughter|child)\b"
    r"|\b(?:honou?r|remember|commemorate|mourn|never\s+forget|say\s+their\s+names?)\b"
    r".{0,180}\b(?:victims?|fallen|killed|murdered|lives?|life|names?|memory|heroes|families)\b"
    r"|\b(?:victims?|fallen|killed|murdered|lives?|life|names?|memory|heroes|families)\b"
    r".{0,180}\b(?:honou?r|remember|commemorate|mourn|never\s+forget|say\s+their\s+names?)\b"
    r"|\b(?:would\s+still\s+be\s+alive|life\s+was\s+taken|lives?\s+were\s+taken|"
    r"preventable\s+(?:murder|death|tragedy)|killed\s+in\s+the\s+line\s+of\s+duty|"
    r"gave\s+everything)\b"
)
```

scripts/tag_lexical.py:1777-1845

```
    for match in PATTERN_THEME_RELIGION.finditer(text):
        start, end = match.span()
        window = text[
            max(0, start - RELIGION_EXPLETIVE_WINDOW_CHARS) : min(
                len(text), end + RELIGION_EXPLETIVE_WINDOW_CHARS
            )
        ]
        if PATTERN_THEME_RELIGION_EXPLETIVE.search(window):
            continue
        return match
    return None


def _coded_nativism_match(text: str, entries: list[dict[str, Any]]) -> re.Match[str] | None:
    """Match coded inheritance/inheritor language only with nearby context.

    Standalone "inheritance" can be about taxes or probate. In the corpus,
    though, inheritance/inheritor language next to nation, citizenship,
    Homeland, birthright, American workers, or immigration terms is a
    nativism signal.
    """
    existing = {str(e["tag"]) for e in entries}
    strong_tag_context = any(
        tag in {"subject:native-born-citizen", "theme:homeland", "theme:nativism"}
        or tag.startswith(("action:", "origin:"))
        or tag
        in {
            "agency:ICEgov",
            "agency:CBP",
            "agency:DHSgov",
            "slogan:criminal-illegal-alien",
            "slogan:illegal-alien",
            "topic:immigration",
        }
        for tag in existing
    )
    for match in PATTERN_NATIVISM_INHERITANCE.finditer(text):
        start, end = match.span()
        window = text[max(0, start - 160) : min(len(text), end + 160)]
        if strong_tag_context or PATTERN_NATIVISM_INHERITANCE_CONTEXT.search(window):
            return match
    return None


def _martyrdom_match(
    text: str, account_category: str, entries: list[dict[str, Any]]
) -> re.Match[str] | None:
    """Match victim-commemoration frames used as a moral justification.

    "This is our why" is highly distinctive in the tracked core-account
    corpus, but noisy in public replies. We therefore accept that phrase
    directly for tracked core/government/official accounts and require
    nearby victim/memorial context elsewhere.
    """
    if m := PATTERN_MARTYRDOM_CONTEXT.search(text):
        return m

    why = PATTERN_MARTYRDOM_WHY.search(text)
    if not why:
```

```
1836:         return None
1837:     existing = {str(e["tag"]) for e in entries}
1838:     if account_category in {"core", "government", "officials"}:
1839:         return why
1840:     if existing.intersection({"subject:angel-family", "subject:crime-victim", "crime:homicide"}):
1841:         return why
1842:     start, end = why.span()
1843:     window = text[max(0, start - 220) : min(len(text), end + 220)]
1844:     if PATTERN_MARTYRDOM_CONTEXT.search(window):
1845:         return why
```

Tag	Count	How it is produced
`theme:criminal`	2,782 (27.8%)	Config pattern in config/tag_taxonomy.yaml: \b(criminal illegal alien|illegal alien|criminal alien|aggravated felon|convicted (?:for|of))\b
`theme:directive`	420 (4.2%)	Emitted by the namespace code shown above or imported from a vetted media-description sidecar and normalized by normalized_media_tags().
`theme:religion`	329 (3.3%)	Emitted when PATTERN_THEME_RELIGION minus PATTERN_THEME_RELIGION_EXPLETIVE matches the combined text buffer.
`theme:martyrdom`	202 (2.0%)	Emitted when PATTERN_MARTYRDOM_CONTEXT / PATTERN_MARTYRDOM_WHY matches the combined text buffer.
`theme:civil-disturbance`	85 (0.8%)	Emitted when PATTERN_THEME_CIVIL_DISTURBANCE and DISTURBANCE_CITY_PATTERNS matches the combined text buffer.
`theme:homeland`	67 (0.7%)	Emitted when PATTERN_THEME_HOMELAND matches the combined text buffer.
`theme:nativism`	30 (0.3%)	Emitted when PATTERN_THEME_NATIVISM / PATTERN_THEME_NATIVISM_BIRTHRIGHT / coded inheritance rule matches the combined text buffer.
`theme:transgender`	23 (0.2%)	Emitted when PATTERN_THEME_TRANSGENDER matches the combined text buffer.
`theme:pop-culture-reference`	5 (0.0%)	Emitted when PATTERN_THEME_POP_CULTURE_REFERENCE and parody emitters matches the combined text buffer.

topic

Topic tags are broader buckets. Some are direct regexes; topic:immigration is also applied by parent implications and by the sticky default for tracked core/government/official accounts unless non-immigration blockers fire.

scripts/tag_lexical.py:413-424

```
413: # tweet, we suppress the default `topic:immigration`. The corpus is
414: # overwhelmingly about immigration enforcement; defaulting to ON is
415: # strictly higher recall than trying to infer relevance from sparse text.
416: NON_IMMIGRATION_PATTERNS: tuple[re.Pattern[str], ...] = (
417:     re.compile(r"\b(weather|hurricane|tornado|earthquake|wildfire)\b", re.I),
418:     re.compile(r"\b(NCAA|Super Bowl|World Series|playoffs?)\b", re.I),
419:     re.compile(r"\b(birthday|anniversary)\b", re.I),
420: )
421:
422: # A "sticky default" applied to every tweet from these account categories
423: # unless a NON_IMMIGRATION_PATTERN matches. See the `topic:immigration`
424: # entry in `config/tag_taxonomy.yaml`.
```

scripts/tag_lexical.py:1855-1909

```
1855:     "action:deportation": ("topic:immigration",),
1856:     "action:self-deportation": ("topic:immigration",),
1857:     "action:report-immigrants": ("topic:immigration",),
1858:     "agency:ICEgov": ("topic:immigration",),
1859:     "agency:CBP": ("topic:immigration",),
1860:     "agency:DHSgov": ("topic:immigration",),
1861:     "agency:HSI_HQ": ("topic:immigration",),
1862:     "agency:USBPChief": ("topic:immigration",),
1863:     "agency:DeptofWar": ("topic:military",),
1864:     "agency:CENTCOM": ("topic:military",),
1865:     "agency:Southcom": ("topic:military",),
1866:     "agency:USCG": ("topic:military",),
1867:     "agency:USArmyNorth": ("topic:military",),
1868:     "agency:USNationalGuard": ("topic:military",),
1869:     "agency:USNorthernCmd": ("topic:military",),
1870:     "theme:criminal": ("topic:immigration",),
1871:     "genre:lineup": ("topic:immigration",),
1872:     "subject:angel-family": ("theme:martyrdom", "topic:immigration"),
1873:     "subject:crime-victim": ("theme:martyrdom", "topic:immigration"),
1874:     "subject:cbp-home-app": ("topic:immigration",),
1875:     "subject:enforcement-op": ("topic:immigration",),
1876:     "policy:border": ("topic:immigration",),
1877:     "policy:cbp-home": ("topic:immigration",),
1878:     "policy:sanctuary-cities": ("topic:immigration",),
1879:     "policy:worksite-enforcement": ("topic:economy", "topic:immigration"),
1880:     "theme:nativism": ("topic:immigration",),
1881:     "theme:pop-culture-reference": ("topic:immigration",),
1882:     "religion:christianity": ("theme:religion",),
1883:     "slogan:criminal-illegal-alien": ("topic:immigration",),
1884:     "slogan:free-ticket-home": ("topic:immigration",),
1885:     "slogan:go-home": ("topic:immigration",),
1886:     "slogan:illegal-alien": ("topic:immigration",),
1887:     "slogan:mass-deportation": ("topic:immigration",),
1888:     "slogan:masa": ("topic:immigration",),
1889:     "slogan:find-and-kill": ("topic:immigration",),
1890:     "slogan:import-third-world": ("topic:immigration",),
1891:     "legal:birthright-citizenship": ("topic:immigration",),
1892:     "genre:parody": ("theme:pop-culture-reference",),
1893:     "slogan:most-secure-border": ("topic:immigration", "policy:border"),
```

```
    "slogan:catch-release": ("topic:immigration",),
    "slogan:project-homecoming": ("topic:immigration",),
    "slogan:maga": ("topic:general",),
    "slogan:maha": ("topic:general",),
    "slogan:america-first": ("topic:general",),
    "slogan:golden-age": ("topic:general",),
    "slogan:save-america": ("topic:general",),
    "slogan:law-and-order": ("topic:general",),
    "slogan:peace-through-strength": ("topic:general",),
    "slogan:promises-kept": ("topic:general",),
    "phrase:immigrant": ("topic:immigration",),
    "phrase:migrant": ("topic:immigration",),
}
INTRINSIC_PARENT_TOPICS_PREFIXES: tuple[tuple[str, str], ...] = (("origin:", "topic:immigration"),)
PATTERN_EXPLICIT_IMMIGRATION_TOPIC = _compile(
    r"\b(immigration|immigrants?|migrants?|asylum|illegal\s+(?:alien|immigrant)s?|"
```

scripts/tag_lexical.py:1936-2041

```
    if PATTERN_EXPLICIT_IMMIGRATION_TOPIC.search(text):
        add("topic:immigration")
    if (
        any(e["tag"] == "policy:worksite-enforcement" for e in entries)
        and "topic:economy" not in existing_tags
    ):
        add("topic:economy")


# Tag names whose presence on a tweet promotes `topic:immigration` from
# a tentative-by-account default to a confirmed-by-text classification.
# Anything that explicitly references the immigration domain (origin
# country pattern, deportation verb, border keyword, ICE/CBP/DHS handle,
# the criminal-alien frame, etc.) clears the bar.
IMMIGRATION_CONFIRMING_PREFIXES: tuple[str, ...] = (
    "action:",
    "origin:",
    "country:",
    "policy:border",
    "policy:sanctuary",
    "policy:worksite",
    "policy:cbp-home",
    "theme:nativism",
    "theme:criminal",
    "shape:",
    "subject:cbp-home-app",
    "subject:enforcement-op",
)
IMMIGRATION_CONFIRMING_EXACT: frozenset[str] = frozenset(
    {
        "agency:ICEgov",
        "agency:CBP",
        "agency:DHSgov",
        "agency:HSI_HQ",
        "agency:USBPChief",
        "slogan:criminal-illegal-alien",
        "slogan:free-ticket-home",
        "slogan:go-home",
        "slogan:illegal-alien",
        "slogan:mass-deportation",
        "slogan:masa",
        "slogan:most-secure-border",
        "slogan:catch-release",
        "slogan:nice",
        "slogan:project-homecoming",
        "slogan:reportrecon",
        "slogan:worst",
        "phrase:immigrant",
        "phrase:migrant",
    }
)
# Last-ditch keyword check — picks up image-heavy / template-light
# tweets that the namespace rules above don't flag but that still
# clearly read as immigration content ("ICE arrested", "illegal alien",
# "the border", standalone "immigration"). Kept narrow on purpose.
PATTERN_IMMIGRATION_KEYWORD = _compile(
    r"\b(immigration|immigrant|migrant|asylum|illegal alien|the border|"
    r"border patrol|ICE\b|CBP\b)\b"
)


def _has_immigration_signal(entries: list[dict[str, Any]], text: str) -> bool:
    """True if the tweet carries any explicit immigration-domain signal.
    Drives the confirmed-vs-tentative split for `topic:immigration`."""
    for e in entries:
        tag = e["tag"]
        if tag in IMMIGRATION_CONFIRMING_EXACT:
            return True
        for pref in IMMIGRATION_CONFIRMING_PREFIXES:
            if tag.startswith(pref):
                return True
    return bool(PATTERN_IMMIGRATION_KEYWORD.search(text))


def _has_non_immigration_topic(entries: list[dict[str, Any]]) -> bool:
    return any(e["tag"].startswith("topic:") and e["tag"] != "topic:immigration" for e in entries)


def _maybe_immigration_default(
    entries: list[dict[str, Any]],
    account_category: str,
    text: str,
    add: Callable[..., None],
) -> None:
    """Apply `topic:immigration` to every tweet from a tracked-tier
    author unless an obvious non-immigration signal blocks it.
    Confirmed when the tweet carries any explicit immigration-domain
```

```
2023:     marker; tentative otherwise (image-heavy, template-light tweets
2024:     where the author identity is the only signal we have).
2025:
2026:     See the `topic:immigration` entry in `config/tag_taxonomy.yaml` for
2027:     the rationale."""
2028:     if account_category not in IMMIGRATION_DEFAULT_CATEGORIES:
2029:         return
2030:     has_signal = _has_immigration_signal(entries, text)
2031:     if has_signal:
2032:         add("topic:immigration")
2033:         return
2034:     if any(e["tag"] == "event:palestine" for e in entries):
2035:         return
2036:     for pat in NON_IMMI
    ... [truncated in PDF; see source file for full pattern]
```

Tag	Count	How it is produced
`topic:immigration`	9,196 (91.9%)	Emitted by explicit immigration text, by parent implications from narrower immigration tags, or by sticky default for tracked core/government/official accounts unless blockers fire; tentative when only the account default supports it.
`topic:economy`	1,083 (10.8%)	Emitted when PATTERN_TOPIC_ECONOMY matches the combined text buffer.
`topic:general`	971 (9.7%)	Emitted when at least three broad problem-domain regexes match, or implied by broad administration slogans in the parent-topic map.
`topic:military`	658 (6.6%)	Emitted when PATTERN_TOPIC_MILITARY matches the combined text buffer.
`topic:laudatory`	99 (1.0%)	Emitted when PATTERN_TOPIC_LAUDATORY matches the combined text buffer.